Vývoj softvérových aplikácií - prednášky a cvičenia

Igor Kossaczký

28. januára 2025

Abstrakt

Učebný text sa venuje tvorbe viacvrstvových aplikácií s využitím štandardizovaných technológií. Zameriava sa predovšetkým na problematiku objektovo-relačného mapovania a REST služieb. Výklad pokrýva obsah prednášok predmetu *Vývoj softvérových aplikácií*, je doplnený kompletnou implementáciou vzorových projektov a implementačnými zadaniami pre cvičenia.

Obsah

1 Úvod do JPA - práca s jednou tabuľkou **7**
1.1 JDBC . . . 7
1.2 Terminológia JPA . . . 8
1.3 Prvá JPA aplikácia . . . 8
1.4 Generovanie tabuľky z entitnej triedy . . . 10

2 Entity a JPA anotácie **13**
2.1 Štruktúra entitnej triedy . . . 13
2.2 Generované kľúče . . . 13
2.3 Identita entitných objektov . . . 14
2.4 Synchronizácia stavu s databázou . . . 15
2.5 Generovanie entitnej triedy z tabuľky . . . 15
2.6 JPQL dotazy . . . 15
2.7 JPA controller . . . 16

3 Jednosmerné asociácie a kolekcie **19**
3.1 Kolekcia objektov jednoduchého dátového typu . . . 19
3.2 Jednosmerná N:N asociácia . . . 20
3.3 Jednosmerná N:1 asociácia . . . 21
3.4 Jednosmerná 1:N asociácia . . . 21

4 Obojsmerné asociácie **25**
4.1 Obojsmerná 1:N - N:1 asociácia . . . 25
4.2 Obojsmerná N:N asociácia . . . 26
4.3 Stratégia dotazovania . . . 27

5 Špecifické situácie **33**
5.1 Kompozícia . . . 33
5.2 Asociácia 1:1 . . . 34
5.3 Mapovanie dvoch tried do jednej tabuľky . . . 34
5.4 Mapovanie jednej triedy do viacerých tabuliek . . . 35
5.5 Zložené kľúče . . . 36

6 Mapovanie hierarchie tried **40**
6.1 Stratégie mapovania . . . 40
6.2 Mapovanie abstraktnej rodičovskej triedy . . . 42
6.3 Výber stratégie . . . 42

7 REST - Úvod **46**
7.1 REST - základné pojmy . . . 46
7.2 Prvá REST aplikácia . . . 47
7.3 REST služba s viacerými zdrojmi . . . 49

8 REST a JPA **53**
8.1 Vytváranie a odstraňovanie zdrojov . . . 53
8.2 Zdroje so štrukturovanými dátami . . . 54
8.3 Integrácia REST a JPA . . . 56

9 Otázky a problémy pri použití REST **60**
9.1 Služba z pohľadu klienta . . . 60
9.2 REST služba s komplexnou štruktúrou zdrojov . . . 61

10 HATEOAS **67**

Úvod

Desaťročia vývoja informačných systémov ukázali, že pri ich tvorbe je možné použiť viaceré univerzálne, od oblasti použitia nezávislé, princípy a postupy. Veľká časť softvérových aplikácií v súčasnosti je postavená na koncepte viac-vrstvovej architektúry. Prináša viacero výhod: Jednotlivé vrstvy môžu bežať samostatne na vlastnej hardvérovej infraštruktúre, nezávisle ich môžu vyvíjať rôzne tímy s využitím špecifických technológií.

Pre webové (ale aj iné) aplikácie poskytujúce používateľovi bohaté používateľské rozhranie je typická 3-vrstvová architektúra pozostávajúca z prezentačnej, aplikačnej a dátovej (perzistentnej) vrstvy. Používateľ komunikuje s prezentačnou vrstvou, ktorá predstavuje používateľské rozhranie. Údaje zadávané používateľom sú spracované algoritmami aplikačnej vrstvy a trvalo uložené a manažované v dátovej vrstve. Spôsob reprezentácie údajov v jednotlivých vrstvách je determinovaný špecifickými požiadavkami danej vrstvy:

- Pre prezentačnú vrstvu je typické používanie hierarchicky štrukturovaných formátov ako json, xml či html, čo korešponduje štruktúre používateľského rozhrania, tvoreného systémom do seba vložených komponentov.
- Na druhej strane, pre perzistentné uloženie dát sa najbežnejšie používajú relačné databázy pre ich schopnosť zabezpečiť konzistentnosť a integritu dát.
- A nakoniec, pri implementácii algoritmov v aplikačnej vrstve sa najčastejšie uplatňuje objektovo orientovaný prístup, kde sú údaje reprezentované ako inštancie dátových tried.

Vrstvy sú navzájom voľne spojené univerzálnymi rozhraniami, zabezpečujúcimi prenos údajov medzi nimi. Keďže úlohy týchto rozhraní sú univerzálne, existujú aj štandardné postupy, nástroje a komponenty, ktoré možno použiť pri vývoji akejkoľvek aplikácie (systému) postavenej na viacvrstvovej architektúre, nezávisle od oblasti použitia.

Hlavným cieľom predmetu Vývoj softvérových aplikácií (skrátene VSA) je oboznámiť študentov s vývojom viacvrstvových aplikácií s využitím štandardizovaných technológií a softvérových komponentov pre rozhrania medzi vrstvami. Konkrétne prvá časť textu, kapitoly 1 - 6, je venovaná princípom objektovo-relačného mapovania (skrátene ORM), štandardu Java Persistence API (skrátene JPA) a technológiám poskytujúcim podporu pre vývoj databázových aplikácií. Druhá časť, kapitoly 7 - 10, sa venuje problematike REST služieb, ktoré sú dnes zrejme najpoužívanejšou technológiou pre výmenu dát medzi prezentačnou a aplikačnou vrstvou.

Učebný text pokrýva obsah prenášok predmetu VSA. Vychádza z dokumentov Java EE [4] resp. Jakarta EE [3], ale aj ďalších informačných zdrojov dostupných na webe. Výklad je založený na jednoduchých implementačných príkladoch ilustrujúcich rôzne aspekty tvorby databázových a webových aplikácií. Dôraz je kladený najmä na problematiku súvisiacu s prenosom a transformáciou údajov medzi vrstvami. Pri špecifikácii príkladov využívame relačné aj UML diagramy. Projekty s kompletnou implementáciou ilustračných príkladov sú k dispozícii na google disku STU[1]. Každá kapitola je doplnená implementačnými zadaniami pre cvičenia. Tieto cvičenia sú neoddeliteľnou súčasťou výkladu, obsahujú doplňujúce informácie, pomáhajú študentovi porozumieť problematike hlbšie a umožňujú látku zvládnuť aj formou samoštúdia. Možnosť ďalej si prehĺbiť poznatky poskytuje množstvo literatúry. Problematike ORM sa venujú napríklad publikácie [5], [2], REST službám [1] a [7].

Problematika ORM je obecná - týka sa rovnako všetkých objektovo-orientovaných jazykov. Technológie pre objektovo-relačné mapovanie, založené na rovnakých princípoch ako JPA, možno nájsť v ekosystéme všetkých objektovo-orientovaných jazykov. To isté možno povedať aj o podpore programovacích jazykov pre vývoj REST služieb.

V učebnom texte ako aj pri výuke používame na implementáciu ilustračných príkladov a zadaní programovací jazyk Java. V nasledujúcej časti úvodnej kapitoly je zhrnutý stručný popis vývojového prostredia a konvencií používaných pri špecifikácii zadaní, interpretácia entitno-relačných (ER) a UML diagramov [6].

Vývojové prostredie

Ilustračné aplikácie sú implementované pomocou nástroja na správu projektov Maven[2] na platforme JDK 21.

Pre vývoj je možné použiť akékoľvek vývojové prostredie pre jazyk Java s podporou nástroja Maven. Na prednáškach budeme používať prevažne prostredie Apache NetBeans 23[3]. Je voľne dostupné, navyše

[1] https://drive.google.com/file/d/1GQQ0JOcZ_HpPrwyEUXSgR1-hbkpZXEOJ/view?usp=drive_link
[2] https://maven.apache.org/
[3] https://netbeans.apache.org/front/main/index.html

poskytuje dobrú podporu pre prácu s databázou, integráciu s aplikačnými servermi a mnoho nástrojov pre generovanie kódu.

Obdobnú funkcionalitu však ponúkajú aj ďalšie vývojové prostredia pre Javu (Intellij, Eclipse,...). Maven projekty je možné bez problémov medzi nimi prenášať. Študenti si teda prostredie pre prácu môžu zvoliť podľa vlastných preferencií, prípadne pracovať priamo s nástrojom Maven z príkazového riadku.

Ako databázu je možné použiť ktorúkoľvek relačnú databázu (poskytujúcu JDBC zavádzač). V ukážkových projektoch budeme prevažne používať PostgreSQL[4], ktorú považujeme za referenčnú databázu, prípadne Derby DB, ktorá je súčasťou prostredia NetBeans. Pre objektovo-relačné mapovanie budeme používať komponent EclipseLink[5] (verzia 3, príp. vyššia), čo je referenčná implementácia štandardu JPA.

REST služby budeme vyvíjať ako webové aplikácie na platforme Jakarta EE 11[6] a nasadzovať na aplikačný server Eclipse Glassfish[7] (verzia 7, príp. vyššia).
Upozornenie: Prenesenie ilustračných príkladov na iný aplikačný server, resp. inú verziu platformy (napr. JavaEE) si môže vyžadovať aj zmeny v zdrojovom kóde.

Maven závislosti

- Pre prístup k databáze prostredníctvom JPA je potrebné do projektu samostatnej (standalone) aplikácie pridať nasledujúce závislosti:

```
<!--JPA a EclipseLink-->
    <dependency>
        <groupId>jakarta.persistence</groupId>
        <artifactId>jakarta.persistence-api</artifactId>
        <version>3.2.0</version>
    </dependency>
    <dependency>
        <groupId>org.eclipse.persistence</groupId>
        <artifactId>org.eclipse.persistence.jpa</artifactId>
        <version>4.0.4</version>
    </dependency>
<!--databazovy driver pre postgres-->
    <dependency>
        <groupId>org.postgresql</groupId>
        <artifactId>postgresql</artifactId>
        <version>42.7.3</version>
    </dependency>
```

- Pre implementáciu REST služby je potrebné do projektu webovej aplikácie vložiť nasledujúce závislosti:

```
<!-- JakartaEE -->
    <dependency>
        <groupId>jakarta.platform</groupId>
        <artifactId>jakarta.jakartaee-api</artifactId>
        <version>11.0.0-M1</version>
        <scope>provided</scope>
    </dependency>
<!-- JAXB -->
    <dependency>
        <groupId>jakarta.xml.bind</groupId>
        <artifactId>jakarta.xml.bind-api</artifactId>
        <version>3.0.1</version>
    </dependency>
```

[4] https://www.postgresql.org/
[5] https://eclipse.dev/eclipselink/
[6] https://jakarta.ee/specifications/platform/11/
[7] https://projects.eclipse.org/projects/ee4j.glassfish

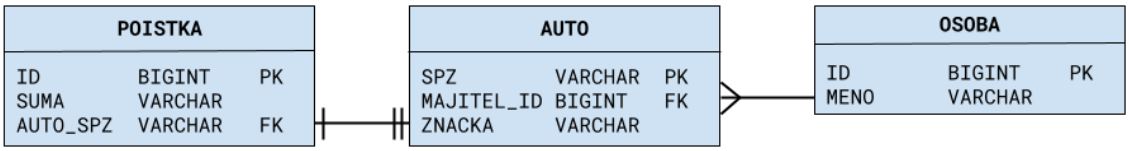
- Závislosti pre implementáciu klienta REST služby:

Listing 1: závoslosti pre klienta REST služby

```
    <dependency>
        <groupId>org.glassfish.jersey.core</groupId>
        <artifactId>jersey-client</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>jakarta.xml.bind</groupId>
        <artifactId>jakarta.xml.bind-api</artifactId>
        <version>3.0.1</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-moxy</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-jaxb</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-json-binding</artifactId>
        <version>3.1.9</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.inject</groupId>
        <artifactId>jersey-hk2</artifactId>
        <version>3.1.9</version>
    </dependency>
```

Interpretácia relačných diagramov

Pre popis štruktúry databázy budeme používať diagramy s tzv. *"crows' foot"* notáciou, ako ilustruje nasledujúca ukážka:



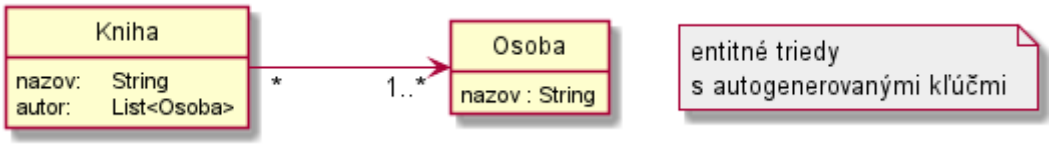
- Primárne kľúče sú v diagrame označené skratkou **PK**, cudzie kľúče **FK**.
- Hrana medzi tabuľkami s rozvetveným zakončením ***"paprčkami"*** označuje **viacnásobnú asociáciu**: *Jedna osoba môže mať viac áut.*
- **Symbol |** označuje **jednoduchú asociáciu** (t.j. FK stĺpec musí byť unikátny). *Auto môže mať najviac jedno poistenie.*
- **Symbol ||** na opačnej strane asociácie hovorí, že cudzí kľúč je **povinný** (`notNULL`). *Poistenie musí byť asociované s nejakým autom.*
- Kľúče môžu byť generované databázou (autogenerované). Táto skutočnosť však nie je v diagrame nijako zvýraznená. Ak má byť kľúč tabuľky autogenerovaný, bude to uvedené explicitne v špecifikácii.

Poznámka: *V ER-diagramoch sa zvykne tiež používať symbol kruhu ○ pre voliteľné údaje. My budeme namiesto vyznačovania nepovinných FK stĺpcov, označovať povinné FK stĺpce symbolom ||. Teda diagram hore hovorí, že poistka musí mať zadanú ŠPZ auta, ale auto nemusí mať zadaného majiteľa.*

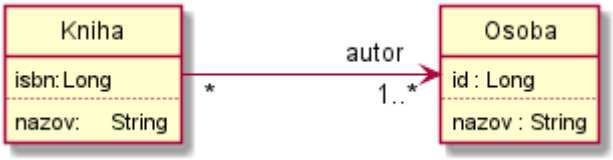
Interpretácia UML diagramov tried

Pre popis objektového modelu budeme používať UML diagramy tried.
Kľúčové atribúty entitných tried, t.j. dátové členy triedy odpovedajúce primárnym kľúčom v tabuľke by mali byť uvedené v popise diagramu alebo zobrazené v triede ako prvé a oddelené od ostatných prerušovanou čiarou.

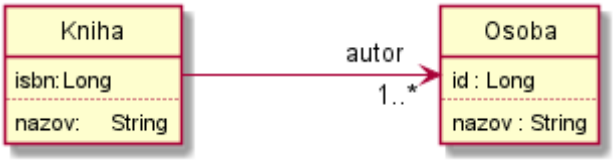
Ak je zo špecifikácie zrejmé, že entita má **autogenerované kľúče**, nemusí byť kľúčový dátový člen pre autogenerovaný kľúč v UML diagrame vôbec zobrazený. Ak nie je uvedené inak, predpokladáme, že autogenerovaný kľúč má názov id a je typu Long.



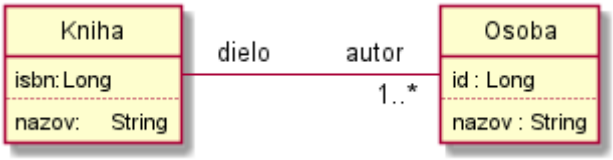
Referencie na asociované objekty sa nezvyknú uvádzať medzi dátovými členmi triedy. Názov referencie udáva popis koncového bodu asociácie.



Násobnosť asociácie * resp. 0..* nemusíme vyznačiť (je to default)



Obojsmerné asociácie sú vyznačené ako hrany bez šípky a mali by mať popis na oboch koncoch. Udávajú totiž názvy referencií na oboch stranách asociácie.



Pozri tiež dokumentáciu k diagramom tried v UML 2 na stránke developer.ibm.com[8]

[8] https://developer.ibm.com/articles/the-class-diagram/

1 Úvod do JPA - práca s jednou tabuľkou

Prácu s jednou databázovou tabuľkou si ilustrujeme na aplikácii, ktorá číta údaje z databázy.

Najskôr aplikáciu implementujeme pomocou nízkoúrovňového **JDBC API**. Následne ju reimplementujeme s využitím objektovo relačného mapovania pomocou **Java Persistance API** (JPA). Ukážeme pri tom konfiguráciu pripojenia, základnú štruktúru entitnej triedy (trieda mapovaná na databázu), vytvorenie manažéra entít (*entit manager*) a ilustrujeme jeho použitie. Nakoniec ukážeme, ako je možné pomocou JPA vygenerovať databázové tabuľky k existujúcim entitným triedam.

Na projektoch demonštrujeme aj použitie rôznych vývojových prostredí a relačných databáz. Pre vytvorenie databázových tabuliek a ich naplnenie údajmi pre testovanie aplikácií môžeme použiť akéhokoľvek databázového klienta.

Prostredie NetBeans má databázového klienta integrovaného v záložke *Services/Databases*.

1.1 JDBC

JDBC - Java Database Connectivity je základné aplikačné programovacie rozhranie pre prístup k relačným databázam v jazyku Java. Prácu s ním ilustruje nasledujúci príklad.

Príklad 1: Jednoduchá aplikácia používajúca priamo JDBC

Aplikácia bude čítať údaje jednoduchej tabuľky, ktorú si pripravíme vopred pomocou databázového klienta:

KNIHA		
NAZOV	VARCHAR	PK
CENA	DOUBLE	

Pre prácu s databázou potrebuje aplikácia načítať databázový zavádzač a vytvoriť pripojenie na databázu:

```
Class.forName("org.postgresql.Driver");
Connection con = DriverManager.getConnection(
      "jdbc:postgresql://localhost:5432/edudb", "edu", "edu4edu");
```

Následne môže volať príkazy v jazyku SQL:

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM kniha");
```

Dotaz vracia generickú dátovú štruktúru ResultSet, ktorou môžeme postupne prechádzať a pristupovať k načítaným záznamom a položkám.

```
rs.next();
System.out.println(rs.getString("nazov");
```

Ak chceme s knihami pracovať ako s objektmi, môžeme vytvoriť jednoduchú dátovú triedu (tzv. pojo - plain old java object[9])

```
public class Kniha {
  String nazov;
  double cena;
}
```

a záznamy z generickej dátovej štruktúry 'manuálne' preklopiť do zoznamu objektov tejto triedy:

```
List<Kniha> zoznam = new ArrayList<>();
while (rs.next()) {
  Kniha k = new Kniha();
  k.nazov = rs.getString(1);          // 1. stlpec nacita a ulozi ako retazec
  k.cena = rs.getInt(2);              // 2. stlpec nacita a ulozi ako cislo
  zoznam.add(k);
}
```

Implementáciu príkladu 1 nájdete v projekte **p1jdbc** na disku STU.

[9] https://www.baeldung.com/java-pojo-class

1.2 Terminológia JPA

Java Persistence API (JPA) je štandard, ktorý špecifikuje vlastnosti a funkcionalitu objektovo-relačného mapovania (ORM) pre programovací jazyk Java. Štandard definuje a používa nasledujúce pojmy:

Persistence Unit

Parametre nevyhnutné pre pripojenie a prácu s databázou sú v JPA reprezentované komponentom nazývaným `Persistence unit`. Jeho konfigurácia sa nachádza v súbore `persistence.xml`.

V NetBeans môžeme súbor `persistence.xml` vytvoriť nástrojom *New..Persistence Unit* v kontextovom menu projektu.

Entité triedy

Dátové triedy, ktorých inštancie reprezentujú entity (záznamy) relačnej databázy, nazývame **entitné triedy**.

Terminologická poznámka: Inštancie entitnej triedy budeme nazývať aj stručne **inštancie entity** alebo **entitné objekty**.

Entity manager

Manažér entít je komponent, ktorý zabezpečuje pripojenie na databázu a poskytuje funkcie pre prácu s ňou. Synchronizuje stav entitných objektov s entitami (záznamami) v databáze.

Dátový objekt, ktorý je manažérom entít prepojený (namapovaný) na entitu v databáze, nazývame **manažovaný objekt**.

Kolekcia všetkých objektov manažovaných manažérom entít sa nazýva **Persistence context**. Pre každý databázový záznam môže persistence context v jednom okamihu obsahovať naviac jednu manažovanú inštanciu.

1.3 Prvá JPA aplikácia

Príklad 2: Reimplemenácia príkladu 1 pomocou JPA

Vytvorenie entitnej triedy

Minimálne požiadavky, ktoré JPA kladie na entitnú triedu, sú:

- spĺňa konvencie *java bean*
  - dátové členy sú *java bean property*, teda sú verejne prístupné len prostredníctvom *getter a setter* metód.
  - konštruktor bez argumentov (môže byť aj default)
- anotácia triedy `@Entity`
- anotácia `@Id` pre dátový člen, ktorý odpovedá kľúčovému stĺpcu.

Dátovú triedu `Kniha` možno teda upraviť na entitnú pridaním uvedených anotácií a prístupových metód:

```
@Entity
public class Kniha {
    @Id
    private String nazov;
    private double cena;

    // public access methods
    public String getNazov() { return nazov; }
    public void setNazov(String nazov) { this.nazov = nazov; }
    public double getCena()  {return cena; }
    public void setCena(double cena) { this.cena = cena; }
}
```

Žiadne ďalšie úpravy nie sú potrebné.

Všimnite si, že **názov triedy je zhodný s názvom tabuľky**, takisto **názvy dátových členov sa zhodujú s názvami stĺpcov**. Vďaka tomu dokáže JPA automaticky priradiť (namapovať) triedu tabuľke a dátové členy jej stĺpcom.

Pripojenie k databáze

Údaje pre pripojenie k databáze zadáme do elementu `persistence-unit`, ktorý sa nachádza v súbore `persistence.xml`. V podelemente `class` je tiež potrebné uviesť názvy entitných tried mapovaných na túto databázu.

```
<persistence-unit name="vsaPU" transaction-type="RESOURCE_LOCAL">
  <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
  <class>vsa.Kniha</class>
  <properties>
    <property name="jakarta.persistence.jdbc.url"
              value="jdbc:postgresql://localhost:5432/edudb"/>
    <property name="jakarta.persistence.jdbc.user" value="edu"/>
    <property name="jakarta.persistence.jdbc.driver"
              value="org.postgresql.Driver"/>
    <property name="jakarta.persistence.jdbc.password" value="edu4edu"/>
  </properties>
</persistence-unit>
```

Pripojenie a prístup k databáze zabezpečuje v aplikácii manažer entít. Vytvoríme ho pomocou faktory metódy, ktorej ako argument zadáme meno elementu `persistence-unit`. V našom príklade: `"vsaPU"`.

```
EntityManagerFactory emf = Persistence
                              .createEntityManagerFactory("vsaPU");
EntityManager em = emf.createEntityManager();
```

Čítanie údajov

Manažér entít poskytuje, okrem iných, aj metódu `createNativeQuery`, ktorá umožňuje vytvoriť `SQL` dotaz na čítanie údajov.

```
Query q = em.createNativeQuery("select * from kniha", Kniha.class);
List<Kniha> knihy = q.getResultList();

for (Kniha k : knihy) {
  System.out.println("" + k);
}
```

Implementáciu príkladu 2 nájdete v projekte **p1jpa** na disku STU.

Čítanie údajov je v projekte implementované funkciou `citajKnihy`. Projekt je doplnený aj o funkcie `zmenCeny`, `najdiKnihu`, `vytvorKnihu` ilustrujúce prácu s transakciami a metódy manažéra `find` a `persist`:

Modifikácia dát

```
Query q = em.createNativeQuery("select * from kniha", Kniha.class);
List<Kniha> knihy = q.getResultList();
em.getTransaction().begin();
for (Kniha k : knihy) {
  k.setCena(k.getCena() * 0.8);
}
em.getTransaction().commit();
```

Vyhľadávanie podľa kľúča

```
Kniha k = em.find(Kniha.class, "Pipi");
System.out.println(k);
```

Vytvorenie nového záznamu

```
Kniha k = new Kniha();
k.setNazov("Brave new world");
em.getTransaction().begin();
em.persist(k);
em.getTransaction().commit();
```

1.4 Generovanie tabuľky z entitnej triedy

JPA umožňuje generovať databázové tabuľky automaticky na základe štruktúry entitných tried. Ilustrujeme to na projekte **p1jpa**. Projekt, prípadne jeho kópiu, najskôr modifikujeme tak, aby aplikácia pracovala s databázou DerbyDB:

- do `pom.xml` pridáme Maven závislosť pre zavádzač databázy DerbyDB.

```
    <dependency>
        <groupId>org.apache.derby</groupId>
        <artifactId>derbyclient</artifactId>
        <version>10.15.2.0</version>
    </dependency>
```

- do `persitence.xml` pridáme resp. upravíme konfiguračné údaje pre pripojenie k databáze.

```
  <persistence-unit name="derbyPU" transaction-type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.jpa.PersistenceProvider</provider>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
        value="jdbc:derby://localhost:1527/sample"/>
      <property name="jakarta.persistence.jdbc.user" value="app"/>
      <property name="jakarta.persistence.jdbc.driver"
        value="org.apache.derby.client.ClientAutoloadedDriver"/>
      <property name="jakarta.persistence.jdbc.password" value="app"/>
      <property name="jakarta.persistence.schema-generation.database.action"
        value="create"/>
    </properties>
  </persistence-unit>
```

Kedže v tejto databáze zatiaľ tabuľka `KNIHA` neexistuje, pridali sme do konfigurácie element `schema-generation.database.action` s hodnotou `create`. Vďaka tomu JPA vygeneruje tabuľku automaticky pri spustení aplikácie.

Stratégie pre generovanie schémy databázy

Element `schema-generation.database.action` môže obsahovať hodnotu:

- `none` Žiadna akcia.
- `create` Ak v databáze neexistuje tabuľka pre entitnú triedu, JPA ju vytvorí ešte pred spustením aplikácie.
- `drop-and-create` Pred spustením najprv odstráni existujúce tabuľky (aj s obsahom) a následne vytvorí nové prázdne tabuľky. Táto voľba je vhodná pre testovanie a ladenie:
  - pri opakovanom spustení aplikácie sa do databázy môžu vkladať znovu tie isté záznamy, nenastane pri tom `unique-key-constraint violation`.
  - pri zmene štruktúry entitnej triedy sa prispôsobí aj štruktúra tabuľky.

Cvičenia

V nasledujúcich úlohách vytvárame rôzne modifikácie projektu **p1jpa** z príkladu 2 .

Cvičenie 1.1 Otvorte projekt **p1jpa** a pracujte s ním v rôznych vývojových prostrediach.
Poznámka: Prostredia musia podporovať Maven projekty.

Cvičenie 1.2 Použite pre projekt **p1jpa** inú relačnú databázu, napr. mySQL.

Cvičenie 1.3 Cieľom cvičenia je zistiť, čo sa stane, ak entitná trieda a tabuľka nemajú rovnakú štruktúru.

V projekte urobte nasledovné úpravy:

- Do tabuľky `KNIHA` pridajte nový stĺpec `ISBN` typu `VARCHAR`. Do triedy `Kniha` však dátový člen pre ISBN nedávajte.
- Stratégiu pre generovanie schémy ponechajte `none`
- Do triedy `Kniha` pridajte nový dátový člen `String autor`. Do tabuľky `KNIHA` však stĺpec pre autora nedávajte.
- V aplikácii volajte funkcie `citajKnihy()`,`vytvorKnihu()` a vysvetlite, čo sa stane.
- V triede `Kniha` dajte dátovému členu `autor` anotáciu `@Transient` a opäť spustite funkciu na čítanie kníh z databázy. Čo sa stane teraz?
  Vysvetlenie: Dátový člen entitnej triedy, ktorý je anotovaný `@Transient`, nebude namapovaný na žiadny stĺpec tabuľky.

Cvičenie 1.4 Vygenerovanie databázy nanovo po zmene štruktúry entitnej triedy.

- Do triedy `Kniha` pridajte nový člen `String autor`. (Tabuľku `KNIHA` ponechajte bez zmien, má len stĺpce `nazov, cena`)
- Stratégiu pre generovanie schémy nastavte na `drop-and-create`.
- Použite funkciu na vloženie novej knihy do databázy.
  Presvedčte sa, že tabuľka `KNIHA` bola nanovo vytvorená a má aktualizovanú štruktúru.

Cvičenie 1.5 Mapovanie dátových členov enumeračného typu.

- Vytvorte enumeračný typ popisujúci dostupnosť knihy
  Do triedy `Kniha` pridajte dátový člen enumeračného typu popisujúceho dostupnosť.

```
enum Dostupnost { NASLKADE, NAOBJEDNAVKU, VYPREDANA };
private Dostupnost dostupnost;
```

  (Nezabudnite pridať aj prístupové metódy)
- Stratégiu pre generovanie schémy nastavte na `drop-and-create`.
  V aplikácii vytvorte novú knihu, zadajte jej dostupnosť a knihu vložte do databázy.
  Pozrite v databáze, aký typ dát obsahuje nový stĺpec `DOSTUPNOST`.
- Dátovému členu `dostupnost` pridajte anotáciu `@Enumerated(EnumType.STRING)`
  Spustite aplikáciu znovu a overte, aký typ dát obsahuje stĺpec `DOSTUPNOST` teraz.
- Vyskúšajte tiež, čo bude obsahovať stĺpec, ak v novej knihe hodnotu pre dostupnosť nezadáte.

Cvičenie 1.6 Rozdiely v mapovaní dátových členov typu `double` a triedy `Double`.

- V tabuľke `KNIHA` vytvorte pomocou databázového klienta novú knihu, ktorá nemá zadanú cenu.
  Poznámka: Ak stĺpec `CENA` nemá povolené hodnoty `NULL`, povolte ich (vo vlastnostiach stĺpca).
- Stratégiu pre generovanie schémy nastavte na `none`.
- V aplikácii načítajte novovytvorenú knihu a presvedčte sa, že dátový člen `cena` obsahuje hodnotu `0.0`.
  Vysvetlenie: Dátový člen `cena` je totiž hodnota typu `double` a nie objekt triedy `Double`.
- V triede `Kniha` zmeňte typ dátového člena `cena` na `Double` a načítajte vytvorenú knihu znovu.
  Presvedčte sa, že teraz je `cena` novej knihy `null`.

Cvičenie 1.7 Generovanie stĺpcov s povolenými hodnotami `NULL`.

- Ak tabuľku `KNIHA` vytvoríme z entitnej triedy `Kniha` (pomocou schema-generation `create` alebo `drop-and-create`), stĺpec `CENA` má hodnoty `NULL` povolené.
  Presvedčte sa o tom.
- V triede `Kniha` pridajte cene anotáciu

```
    @Column(nullable=false)
    private double cena;
```

- Stratégiu generovania databázy ponechajte `drop-and-create`.
  Spustite aplikáciu a presvedčte sa, že teraz stĺpec `CENA` nemá povolené hodnoty `NULL`.
- Následne zmeňte typ ceny na Double (nezabudnite upraviť aj prístupové metódy).

```
    @Column(nullable=false)
    private Double cena;
```

Vytvorte v aplikácii novú knihu, ktorá nemá zadanú cenu a poskúste ju vložiť do databázy. Čo sa stane?

Cvičenie 1.8 Unikátne stĺpce.

- Pridajte do triedy `Kniha` dátový člen pre ISBN s anotáciou:

```
    @Column(unique=true)
    private String isbn;
```

- Stratégiu generovania databázy ponechajte `drop-and-create`.
  Spustite aplikáciu a pozrite sa, akým spôsobom sa v štruktúre databázy prejavila unikátnosť stĺpca `ISBN` (mal by byť pre neho vytvorený index s vlastnosťou `unique`).
- Upravte program tak, aby ste si overili, že do databázy nemožno vložiť dve knihy s rovnakým ISBN.

2 Entity a JPA anotácie

V tejto časti sa detailnejšie oboznámime so štruktúrou entitných tried a funkcionalitou manažéra entít. V novom projekte si najprv nástrojom prostredia vygenerujeme entitnú triedu a objasníme:

- generované kľúče
- persistence context a problematiku rovnosti entitných objektov
- metódy manažéra entít
- mapovanie rôznych dátových typov
- unikátne a nepovinné stĺpce

Potom si ukážeme generovanie entitnej triedy z databázovej tabuľky.
Oboznámime sa aj ďalšími JPA anotáciami a dotazovacím jazykom JPQL.

2.1 Štruktúra entitnej triedy

Prostredie NetBeans poskytuje v kontextovom menu projektu viaceré nástroje na vytvorenie typickej entitnej triedy podľa konvencií ORM spĺňajúce požiadavky JPA.

Nástroj *New/Other/Persistence/Entity class...* vytvorí základnú kostru typickej entitnej triedy, ktorá:

- implementuje rozhranie `Serializable`,
- obsahuje len dátový člen `id` pre kľúčový stĺpec tabuľky - má anotáciu `@Id`,
- a metódy `equals, hashCode` a `toString` zdedené z triedy Object.

Ďalšie dátové členy s prístupovými metódami môžeme do triedy pridať ručne. V prostredí NetBeans môžeme použiť tiež nástroj *Add property ...* v kontextovom menu editora.

Príklad 3: Vzorová entitná trieda

V novom projekte vytvoríme nástrojmi prostredia NetBeans. entitnú triedu `Person`. Do triedy pre ilustráciu pridáme dátové členy rôznych typov.

```
    private String name;
    private double salary;
    @Temporal(jakarta.persistence.TemporalType.DATE)
    private Date born;
    private boolean married;
```

Všimnite si:

- Anotáciu `@Temporal`, ktorá je povinná pre dátové členy typu `Date`.
  Určuje dátový typ stĺpca v databáze. Databázy totiž poskytujú rôzne dátové typy pre údaje o čase: `DATE, TIME, TIMESTAMP`.
- Názov prístupovej metódy `isMarried()`.
  Podľa konvencií *java-bean* sa názov *getter* metódy pre dátový člen typu `boolean` tvorí pomocou prefixu `is` namiesto `get`.

2.2 Generované kľúče

V entitnej triede `Person` si všimnite, že kľúčový dátový člen `id` má okrem anotácie `@Id` aj anotáciu `@GeneratedValue`.

Anotácia `@GeneratedValue` zabezpečí, že unikátnu hodnotu kľúčového stĺpca vygeneruje samotná databáza pri vložení nového záznamu. Nie je teda potrebné inicializovať hodnotu kľúča v novovytvorenom objekte.

V príklade 3 to ilustruje jednoduchá aplikácia - metóda `main` v triede `NewMain`. Metóda vytvorí inštanciu triedy `Person`, inicializuje hodnoty dátových členov, však hodnotu pre kľúč nezadá. Následne zavolá metódu `persist` pre vytvorenie nového záznamu v databáze.

Po ukončení transakcie sa môžeme presvedčiť výpisom na obrazovku, že kľúčový dátový člen `id` obsahuje nenulovú hodnotu vygenerovanú databázou.

Poznámka: Všimnite si tiež aký databázový typ majú stĺpce `MARRIED` a `BORN`.

Implementáciu príkladu 3 nájdete v projekte **p21a** na disku STU.

2.3 Identita entitných objektov

Kostra entitnej triedy `Person` vytvorená v projekte **p21a** má už vlastnú implementáciu metód `hashCode` a `equals`.

```
    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Person)) {
            return false;
        }
        Person other = (Person) object;
        if ((this.id == null && other.id != null) || (this.id != null && !this.
            id.equals(other.id))) {
            return false;
        }
        return true;
    }
```

Ako vidíme, implementované sú tak, aby dva objekty boli považované za ekvivalentné (t.j. identické) práve vtedy, keď majú rovnaký kľúč, teda reprezentujú ten istý záznam v databáze.

Pripomeňme, že v relačných databázach nesmú existovať v jednej tabuľke dva záznamy s rovnakým primárnym kľúčom. Na rozdiel od toho, objektovo orientované programovanie umožňuje súčasnú existenciu viacerých inštancií entitnej triedy s rovnakou hodnotou kľúčového dátového člena, ostatné dátové členy môžu mať pri tom aj rôzne hodnoty.

Preto manažér entít môže manažovať t.j. synchronizovať s databázou len jednu z týchto inštancií. Nazývame ju preto manažovaná alebo pripojená (*attached*), ostatné sú nemanažované (*detached*).

Ilustrácia metódy equals a manažovaných objektov

Pre ilustráciu doplníme projekt o jednoduchú metódu, ktorá:

- Najprv vytvorí nový objekt triedy `Person` a
- zavolá `persist` pre jeho vloženie do databázy.
- Následne použije metódu `find`, ktorá vyhľadá podľa kľúča novovytvorený záznam, načíta a vráti späť ako objekt triedy `Person`.
- Nakoniec overí metódou `equals`, že tieto dva objekty sú ekvivalentné, t.j. reprezentujú ten istý záznam v databáze.

Metódu implementujeme v dvoch verziách:

Prvá verzia použije pre metódy `persist` i `find` ten istý manažér entít. Môžete sa presvedčiť, že metóda `find` nenačíta údaje do nového objektu, ale vráti priamo referenciu na pôvodný objekt vytvorený v prvom kroku. V skutočnosti vôbec údaje z databázy nečíta.

Druhá verzia po volaní `persist` použitý manažér zavrie a pre čítanie vytvorí nový. Metóda `find` teraz vráti novú inštanciu. V pamäti budú teda dva rôzne objekty s tým istým kľúčom. Manažovaný, t.j. synchronizovaný s databázou, bude však len druhý.

```
    Person p;
    p = new Person();
    p.setName("Fero");
    EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");

    EntityManager em = emf.createEntityManager();
    em.getTransaction().begin();
    em.persist(p);
    em.getTransaction().commit();
    long id = p.getId();
    System.out.println("id: " + id);
    em.close();

    em = emf.createEntityManager();
    Person p2 = em.find(Person.class, id);
    System.out.println("p == p2 :      " + (p == p2));
    System.out.println("p.equals(p2): " + p.equals(p2));
```

Doplnenú implementáciu príkladu 3 nájdete v projekte **p21b** na disku STU.

2.4 Synchronizácia stavu s databázou

Okrem metód `persist` a `find` poskytuje manažér entít radu ďalších metód na synchronizáciu stavu entitného objektu s databázou:

- `detach` - uvoľní manažovaný objekt z množiny objektov spravovaných manažérom entít,
- `clear` - uvoľní všetky objekty spravované manažérom entít,
- `refresh` - obnoví pôvodný stav manažovaného objektu údajmi z databázy,
- `merge` - aktualizuje stav v databáze údajmi z objektu (nemusí byť manažovaný) a vráti manažovaný objekt s aktualizovaným stavom,
- `remove` - odstráni z databázy záznam so stavom entitného objektu. (Samotný objekt sa stane nemanažovaným, existuje však ďalej.)

Ilustráciu použitia týchto metód nájdete v projekte **p21c** na disku STU.

Poznámka: Projekt obsahuje niekoľko funkcií, ktoré môžeme spúšťať postupne za sebou a používa opäť entitnú triedu `Person`.

2.5 Generovanie entitnej triedy z tabuľky

Vývojové prostredie NetBeans poskytuje v kontextovom menu projektu nástroj *Entity Classes from Database*, ktorým je možné vygenerovať hotovú entitnú triedu pre existujúcu tabuľku.

Poznámka: Tento generátor nie je súčasťou JPA implementácie. Podobnú funkcionalitu môžu poskytovať aj iné vývojové prostredia a nástroje. Generovaný zdrojový kód však nemusí byť identický.

Príklad 4:

V prostredí NetBeans vytvoríme nový projekt a vygenerujeme v ňom entitnú triedu pre tabuľku `PERSON`, s ktorou sme pracovali v predchádzajúcom príklade.

Vygenerovaný zdrojový kód ilustruje viacero ďalších nepovinných, ale užitočných JPA anotácií:

- `@Table(name = "PERSON")` anotácia triedy umožňuje zadať názov tabuľky pre entitnú triedu. Využiť ju teda môžeme, ak je meno tabuľky odlišné od mena triedy.
- `@Column(name = "NAME")` anotácia dátového člena umožňuje zadať názov stĺpca tabuľky pre daný dátový člen.
- `@Basic(optional = false)` anotácia hovorí, že dátový člen objektu nesmie mať hodnotu null. Poznámka: Táto anotácia však neznamená, že hodnoty v databáze sú povinné, a pri generovaní tabuľky z entitnej triedy sa vytvorí 'notNULL' stĺpec. To možno dosiahnuť anotáciou `@Column(nullable=false)`.

2.6 JPQL dotazy

Všimnite si anotácie vygenerovanej triedy `Person`:

```
@Entity
@Table(name = "PERSON")
@NamedQueries({
    @NamedQuery(name = "Person.findAll", query = "SELECT p FROM Person p"),
    @NamedQuery(name = "Person.findById",
                query = "SELECT p FROM Person p WHERE p.id = :id"),
    @NamedQuery(name = "Person.findByBorn",
                query = "SELECT p FROM Person p WHERE p.born = :born"),
    @NamedQuery(name = "Person.findByMarried",
                query = "SELECT p FROM Person p WHERE p.married = :married"),
    @NamedQuery(name = "Person.findByName",
                query = "SELECT p FROM Person p WHERE p.name = :name"),
    @NamedQuery(name = "Person.findBySalary",
                query = "SELECT p FROM Person p WHERE p.salary = :salary")})
public class Person implements Serializable {
```

Anotácie `@NamedQuery` definujú pomenované dotazy pre čítanie údajov z databázy. Pre špecifikáciu dotazu však namiesto SQL používajú jeho 'klon' jazyk **JPQL - *Java Persistance Query Language***.

Jazyk JPQL používa namiesto názvov tabuliek a stĺpcov mená entitných tried a ich dátových členov. V nasledujúcich kapitolách uvidíme, že poskytuje aj ďalšiu objektovo-orientovanú podporu, napr. dotazy nad hierarchiou entitných tried.

Použitie JPQL-dotazov ilustrujú nasledujúce ukážky:

- Jednoduchý pomenovaný dotaz

```
TypedQuery<Person> q = em.createNamedQuery("Person.findAll", Person.class);
List<Person> pl = q.getResultList();
for (Person p : pl) {
    System.out.println(p.getName() + " narodeny=" + p.getBorn());
}
```

- Dotaz s parametrami

```
TypedQuery<Person> q = em.createNamedQuery("Person.findByName",Person.class)
    ;
q.setParameter("name", "Fero");
```

- Jazyk JPQL môžeme použiť aj na vytvorenie **nepomenovaného dotazu** metódou `createQuery`. Vyhľadanie osôb bez dátumu narodenia

```
  TypedQuery<Person> q2 = em.createQuery(
    "select p from Person p where p.born is null", Person.class);
```

- Dotaz s jedinou návratovou hodnotou - zistenie počtu osôb s platom menším ako 1000

```
  TypedQuery<Long> q3 = em.createQuery(
    "select count(p) from Person p where p.salary < 1000", Long.class);
  System.out.println("" + q3.getSingleResult());
```

Implementáciu príkladu 4 nájdete v projekte **p23jpql** na disku STU.

2.7 JPA controller

`JPA controller` je fasáda vytvorená nad manažérom entít, ktorá poskytuje sadu jednoduchých CRUD metód pre prácu s jednou databázovou tabuľkou. Podobné komponenty sú známe aj v mnohých iných prostrediach pod názvom DAO - *data access objects*.

`JPA controller` si pre každú jednu zmenu v tabuľke vytvára vlastnú transakciu, preto jeho použitie nie je vhodné v situáciách, ktoré si vyžadujú rozsiahlejšie transakcie zahŕňajúce zmeny vo viacerých entitách.

V NetBeans je ho možné vygenerovať nástrojom *JPA Controller Classes from Entity Classes…*

Cvičenia

Cvičenie 2.1 Mapovanie rôznych dátových typov.

Modifikujte projekt **p21a** a pozorujte, ako sa prejavia zmeny vo vygenerovanej tabuľke:

- V aplikácii inicializujte aj dátové členy `married` a `born` objektu `Person`.
  Návod pre inicializáciu dátumu:

```
    private static Date toDate(int y, int m, int d) {
        Calendar calendar = Calendar.getInstance();
        calendar.set(y, m, d, 0, 0, 0);
        return calendar.getTime();
    }
```

- Členu `married` dajte anotáciu `@Column(columnDefinition="BOOLEAN")`
- Zmeňte typ člena `born` na `LocalDate` resp. `LocalDateTime` a odstráňte anotáciu `@Temporal`.

Porovnajte správanie pre rôzne typy databáz.

Cvičenie 2.2 Mechanizmus generovania kľúčov.

Databázy môžu poskytovať viacero mechanizmov pre generovanie kľúčov. Voľbu mechanizmu umožňuje argument anotácie `@GenerationType`. Vyskúšajte aplikáciu v projekte **p21a** s rôznymi voľbami mechanizmu a pre rôzne databázy.

- Do súboru persistence.xml pridajte riadky:

```
<property name="eclipselink.logging.level.sql" value="FINE"/>
<property name="eclipselink.logging.parameters" value="true"/>
```

  Aktivuje sa tým logovanie sql-príkazov generovaných manažérom entít.
- Spustite aplikáciu a v logu pozorujte generované SQL príkazy. Všimnite si príkazy potrebné pre generovanie kľúča.
- Zistite, čo sa stane, ak pre tabuľku s generovaným kľúčom zadáme v aplikácii hodnotu kľúčovému dátovému členu. Overte v databáze, či bude mať kľúč zadanú alebo generovanú hodnotu.

Cvičenie 2.3 Použite aplikáciu z projektu **p21a** a porovnajte prácu s generovanými a negenerovanými kľúčmi.

- Stratégiu generovania databázy ponechajte `create`.
- Spustite aplikáciu dvakrát za sebou a presvedčte sa, že v databáze vznikli dva nové záznamy, ktoré sa líšia len hodnotou kľúča.
- V entitnej triede odstráňte anotáciu `@GeneratedValue`.
- Keďže teraz nebude kľúč generovaný databázou, zadajte v aplikácii vhodnú hodnotu aj pre kľúčový dátový člen.
- Spustite opäť aplikáciu dvakrát za sebou (s rovnakým kľúčom). Čo sa stane teraz?

Cvičenie 2.4 Použite projekt **p21b** a sledujte synchronizáciu stavu objektu s databázou.

- Aktivujte logovanie generovaných sql-príkazov.
- Na koniec aplikácie pridajte kód, ktorý zmení plat v oboch objektoch a vykoná transakciu

```
        p2.setSalary(3300.0);
        em.getTransaction().begin();
        p.setSalary(2200.0);
        em.getTransaction().commit();
```

- Spustite aplikáciu a overte v logu, či metóda `find` skutočne číta z databázy.
- V databáze príp. v logu overte, ktorý plat sa zapísal do databázy.

Porovnajte obe verzie aplikácie.

Cvičenie 2.5

V projekte **p21c** aktivujte logovanie, spustite aplikáciu a v logu pozorujte SQL-príkazy generované jednotlivými príkazmi.

Cvičenie 2.6 Mapovanie entitnej triedy na existujúcu tabuľku.

V novom projekte vytvorte entitnú triedu `Osoba` s dátovými členmi

```
    @Id
    private Long id;
    private String meno;
    private double plat;
    private Date narodeny;
    private boolean zenaty;
```

- V anotáciách `@Table` a `@Column` nastavte argument `name` tak, aby bola trieda `Osoba` namapovaná na tabuľku `PERSON` z predchádzajúceho cvičenia.
- Stratégiu pre generovanie nechajte `none`.
- Implementujte aplikáciu, ktorá vytvorí niekoľko osôb s rôznymi údajmi a vloží ich do databázy.

Cvičenie 2.7

Pokračujte v predchádzajúcom cvičení a implementujte ďalšie aplikácie pre čítanie a zápis osôb:

- Pomocou JPQL-dotazu načítajte všetky osoby, zvýšte im plat o 100 a zapíšte zmenu do databázy.
- Pomocou JPQL-dotazu zistite mená osôb, ktoré majú plat nižší ako 1000.0.
- Pomocou JPQL-dotazu zistite celkovú sumu platov všetkých osôb.

3 Jednosmerné asociácie a kolekcie

Nasledujúce tri kapitoly sa venujú rôznym typom asociácií v objektových modeloch a ich mapovaniu do relačných databáz. Pri výklade a implementácii budeme postupovať nasledovne:

- Na základe OO-modelu (UML class diagramu) implementujeme entitné triedy.
- Vytvoríme aplikáciu s funkciou na vkladanie údajov do databázy a príp. ďalšími funkciami na dotazovanie.
- Spustením aplikácie vygeneruje pomocou JPA z entitných tried databázové tabuľky.
- Pozrieme si štruktúru a obsah vytvorených tabuliek.
- Prípadne, na rôznych verziách aplikácie ilustrujeme ďalšie vlastnosti a funkcionalitu.

V tejto kapitole začneme mapovaním kolekcií a následne sa budeme venovať jednosmerným asociáciám.

3.1 Kolekcia objektov jednoduchého dátového typu

Pripomeňme, že prvá normálna forma pre relačné databázy vyžaduje, aby stĺpce obsahovali len atomické hodnoty. Teda hodnoty nesmú byť štruktúrované alebo viacnásobné. Na rozdiel od toho, v objektovo orientovaných jazykoch dátové členy tried nemusia obsahovať len hodnoty jednoduchých dátových typov, ale aj štruktúry alebo kolekcie.

Nasledujúci príklad ilustruje, akým spôsobom JPA mapuje viacnásobný dátový člen jednoduchého dátového typu.

Príklad 5: Kniha so zoznamom mien autorov

Entitná trieda - 1.Verzia

```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
  private List<String> autori;
```

Aplikácia

Tabuľky vytvoríme spustením aplikácie, ktorá vytvorí inštanciu knihy s viacerými autormi a vloží ju do databázy (stratégia pre generovanie schémy: `create`).

Všimnite si, že vo vygenerovanej tabuľke `KNIHA` je stĺpec `AUTORI` typu `BLOB`, t.j. obsahuje binárne dáta - serializovaný zoznam autorov. Jednotlivé položky zoznamu však nebudú prístupné pre štandardného databázového klienta. Problém ilustruje **cvičenie 3.1**.

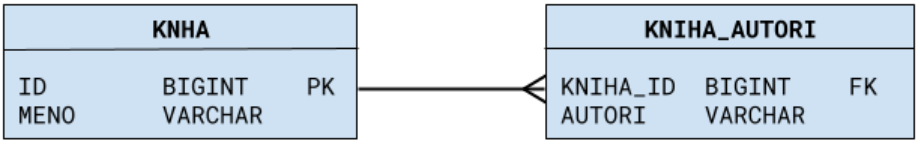
Entitná trieda - 2. Verzia

Zoznamu autorov dáme anotáciu `@ElementCollection`

```
  @ElementCollection
  private List<String> autori;
```

Relačný diagram

Tentokrát sú v databáze vytvorené dve tabuľky:



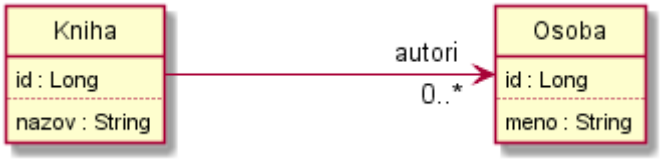
Implementáciu príkladu 5 nájdete v projekte **p31kolekcia** na disku STU.

3.2 Jednosmerná N:N asociácia

N:N asociáciu ilustrujeme na modifikácii predchádzajúceho príkladu, kde budú autori inštancie entitnej triedy `Osoba`.

Príklad 6: Kniha so zoznamom autorov

Kniha môže mať viac autorov a jedna osoba môže byť autorom viacerých kníh. Kniha má zoznam svojich autorov, ale autori nemajú zoznam svojich kníh.



Entitné triedy

```
@Entity
  public class Kniha implements Serializable {
    @Id
    private Long id;
    private String nazov;
    private List<Osoba> autori;
```

```
  @Entity
  public class Osoba implements Serializable {
    @Id
    private Long id;
    private String meno;
```

Poznámka: Na označenie asociácie N:N môžeme použiť anotáciu `@ManyToMany`.

```
    @ManyToMany
    private List<Osoba> autori;
```

V tomto prípade však nie je potrebná, pretože kolekcia referencií na objekty entitnej triedy je automaticky (default) považovaná za N:N asociáciu.

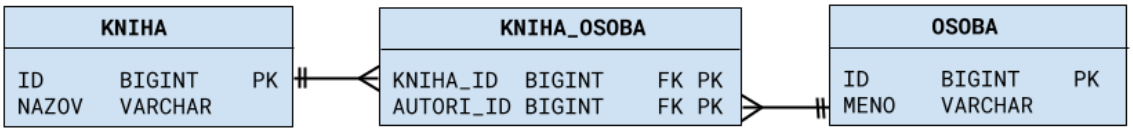
Aplikácia

Vytvoríme inštanciu knihy s viacerými autormi a vložíme ju do databázy. Metódu `persist` treba zavolať aj pre všetky novovytvorené asociované objekty a urobiť commit v tej istej transakcii.

```
    kniha.getAutori().add(autor1);
    kniha.getAutori().add(autor2);

    em.getTransaction().begin();
    em.persist(kniha);
    em.persist(autor1);
    em.persist(autor2);
    em.getTransaction().commit();
```

Relačný diagram

Pre N:N asociáciu je potrebná prepojovacia tabuľka.



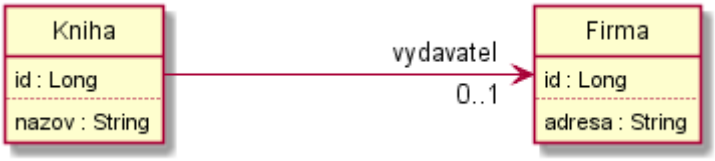
Všimnite si, ako je vytvorený názov prepojovacej tabuľky a jej stĺpcov. Anotáciou `@JoinTable` môžeme tieto názvy definovať sami, pozri cvičenie 3.2.

Implementáciu príkladu 6 nájdete v projekte **p32uniNN** na disku STU.

3.3 Jednosmerná N:1 asociácia

Príkladom jednosmernej asociácie N:1 je vzťah medzi knihou a jej vydavateľom. Trieda `Kniha` má referenciu na firmu, ktorá ju vydáva. Trieda `Firma` však nemá referencie na svoje knihy.

Príklad 7: Kniha s referenciou na jej vydavateľa



Poznámka: Kardinalita 0..1 na strane vydavateľa musí byť vyznačená, pretože inak by nebolo z diagramu jasné, že jedna kniha môže mať len jedného vydavateľa.

Entitné triedy

```
@Entity
  public class Kniha implements Serializable {
    @Id
    private Long id;
    private String nazov;
    private Firma vydavatel;
```

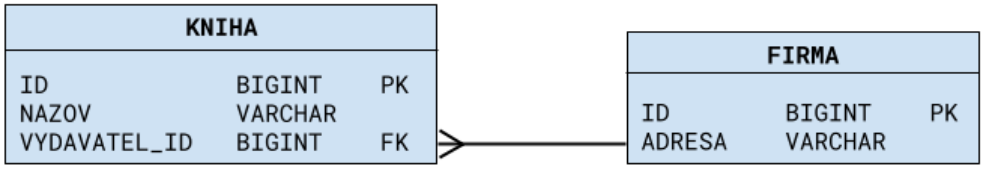
```
@Entity
  public class Firma implements Serializable {
    @Id
    private Long id;
    private String adresa;
```

Poznámka: Na označenie asociácie N:1 môžeme použiť anotáciu @ManyToOne.

```
 @ManyToOne
   private Firma vydavatel;
```

V tomto prípade však nie je potrebná, pretože jednoduchá referencia na objekt entitnej triedy je považovaná automaticky za N:1 asociáciu.

Relačný diagram



Implementáciu príkladu 7 nájdete v projekte **p33uniN1** na disku STU.

3.4 Jednosmerná 1:N asociácia

Otočením orientácie asociácie v predchádzajúcom príklade dostaneme jednosmernú 1:N asociáciu. Tentokrát má teda firma zoznam publikácií, ktoré vydáva, ale kniha nepozná svojho vydavateľa.

Príklad 8: Vydavateľská firma so zoznamom publikácií, ktoré vydáva

Poznámky:

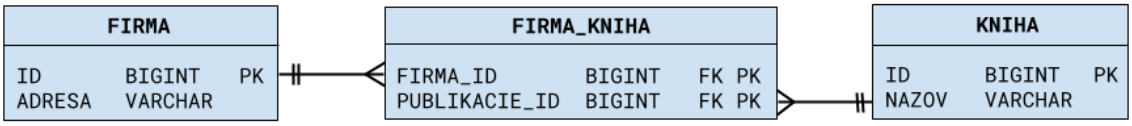
- Hoci trieda Kniha nemá referenciu na Firmu, kardinalita 0..1 na strane firmy je potrebná, aby bolo jasné, že ide o 1:N asociáciu (nie N:N), teda Kniha nemôže mať viac vydavateľov.
- Na strane knihy nie je uvedená kardinalita, to znamená, že je ľubovoľná, teda je 0..*.

Entitné triedy - 1. verzia

```
@Entity
public class Kniha implements Serializable {
  @Id
  private Long id;
  private String nazov;
```

```
@Entity
public class Firma implements Serializable {
  @Id
  private Long id;
  private String adresa;
  private List<Kniha> publikacie;
```

Relačný diagram



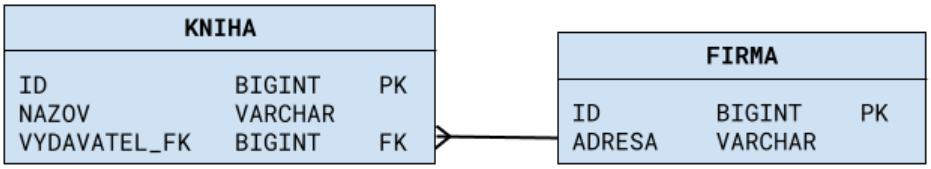
Poznámka: Bola opäť vygenerovaná prepojovacia tabuľka, pretože len na základe entitných tried nie je možné určiť, či asociácia mala byť 1:N alebo N:N (teda, či jedna kniha nemôže mať viac vydavateľov).

Entitné triedy - 2. verzia

```
@Entity
public class Firma implements Serializable {
  @Id
  private Long id;
  private String adresa;
  @OneToMany
  @JoinColumn(name = "VYDAVATEL_FK")
  private List<Kniha> publikacie;
```

Ak nechceme vytvoriť prepojovaciu tabuľku, keďže ide o 1:N asociáciu, môžeme použiť anotáciou `@OneToMany`. Okrem toho však treba definovať názov prepojovacieho stĺpca, ktorý bude v tabuľke asociovanej triedy. To určíme anotáciou `@JoinColumn`.

Relačný diagram



Všimnite si, že teraz má databáza rovnakú štruktúru ako v prípade N:1 asociácie (predchádzajúci projekt). Pri 1:N aj N:1 asociáciách je cudzí kľúč vždy v tabuľke na viacnásobnej strane a referencuje entitu na jednoduchej strane asociácie.

Jediný rozdiel je v názve FK stĺpca, ktorý sme nazvali `VYDAVATEL_FK`, avšak v predchádzajúcom projekte bol vygenerovaný názov `VYDAVATEL_ID`.

Implementáciu príkladu 8 nájdete v projekte **p34uni1N** na disku STU.

Cvičenia

V nasledujúcich úlohách vytvárame rôzne modifikácie projektov z prednášky.

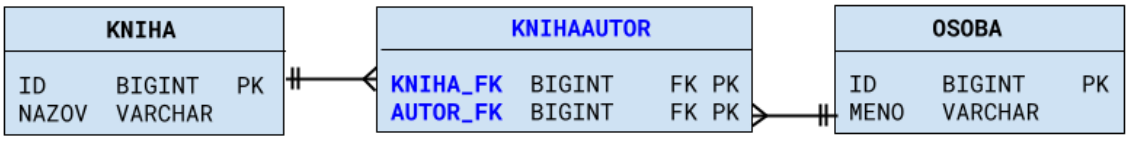
Cvičenie 3.1 Doplňte projekt **p31kolekcia** o dve funkcie:

1. vyhľadá a načíta knihu podľa id (pomocou metódy `find`) a vypíše jej autorov.
2. pomocou JPQL dotazu vyhľadá knihy podľa autora.
   Návod: Skúste dotaz: `"select k from Kniha k where k.autori='Ritchey'"`

Implementujte aplikáciu na otestovanie funkcií. Overte, ako pracujú pre obe verzie entitnej triedy, s anotáciou `@ElementCollection` aj bez nej.

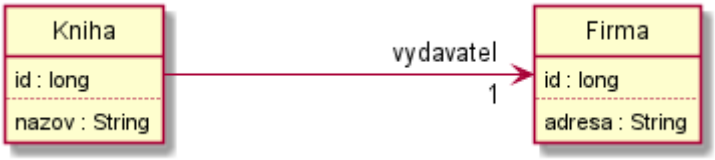
Cvičenie 3.2 Prispôsobenie prepojovacej tabuľky.

V projekte **p32uniNN** upravte entitnú triedu pomocou anotácie `@JoinTable` tak, aby vygenerovaná prepojovacia tabuľka mala štruktúru (pozri www.baeldung.com/jpa-many-to-many[10]):

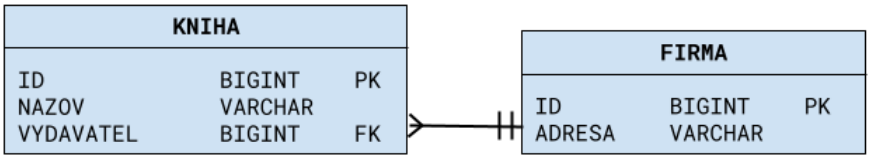


Poznámka: Názvy tried a dátových členov v projekte meniť netreba.

Cvičenie 3.3 V projekte **p33uniN1** upravte entitné triedy podľa UML diagramu,



Anotácie tried upravte tak, aby z nich boli vygenerované databázové tabuľky podľa diagramu:



a) Vydavateľ je povinný atribút, teda vo vygenerovanej tabuľke by mal byť FK stĺpec notNull (čo je znázornené || na relačnom diagrame).
b) Prepojovací stĺpec vo vygenerovanej tabuľke KNIHA sa volá VYDAVATEL (a nie VYDAVATEL_ID)

Cvičenie 3.4 V projekte **p32uniNN** implementujte nasledovné funkcie:

a) `List<Osoba> nájdiAutorov(Long idKnihy),`
   ktorá dostane id knihy, vyhľadá ju a vráti zoznam jej autorov. Ak kniha neexistuje, vráti null.
b) `int pocetKnih(String menoAutora),`
   ktorá dostane meno osoby a vráti počet kníh, ktorých je autorom. Ak osoba s daným menom neexistuje, vráti 0.
c) `List<Kniha> nájdiKnihy(String menoAutora),`
   ktorá dostane meno osoby a vráti zoznam kníh, ktorých je autorom. Ak osoba neexistuje, vráti null.

Pripravte vhodné testovacie dáta a otestujte si funkcie pomocou jednoduchého programu.

Cvičenie 3.5 V projektoch **p33uniN1** aj **p34uni1N** z prednášky implementujte funkciu:

```
void pridajKnihu(String nazov, String adresa)
```

Funkcia dostane názov knihy a adresu firmy, ktorá je jej vydavateľom, a vloží ju do databázy, pričom:

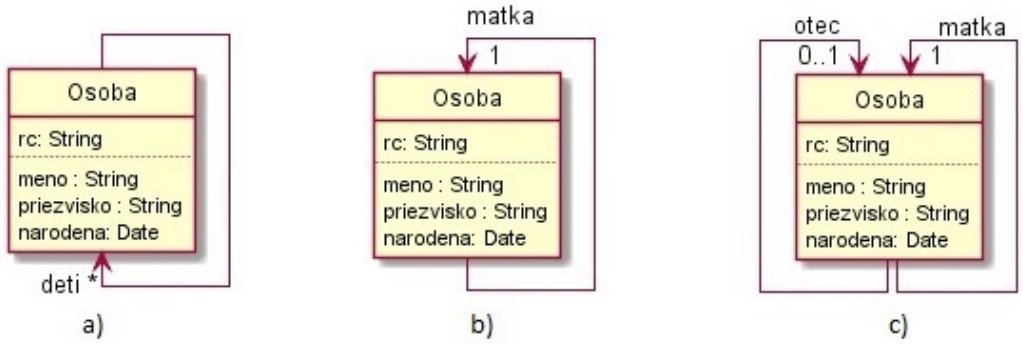
- Najprv overí, či kniha s daným názvom, resp. vydavateľstvo s danou adresou už v DB existuje.
- Ak vydavateľstvo s danou adresou v DB ešte neexistuje, vytvorí ho.
- Ak kniha s daným názvom v DB ešte neexistuje, vytvorí ju a priradí jej zadané vydavateľstvo (zaradí ju medzi publikácie vydavateľstva).
- Ak kniha s daným názvom už v DB existovala, neurobí nič.

[10] https://www.baeldung.com/jpa-many-to-many

Pripravte vhodné testovacie dáta a otestujte si funkciu pomocou jednoduchého programu.

Cvičenie 3.6

Vzťahy medzi deťmi a rodičmi môžeme modelovať rôznymi spôsobmi, ako ukazujú tri rôzne verzie UML diagramu triedy Osoba:



Pre každú verziu:

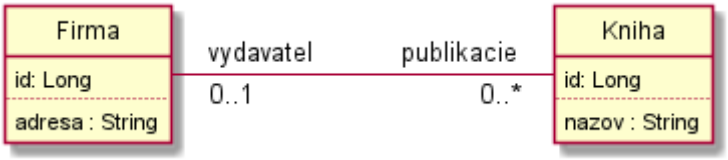
- navrhnite a načrtnite, ako by vyzerala databáza odpovedajúca UML diagramu.
- implementujte entitnú triedu podľa UML diagramu a použite JPA anotácie tak, aby z nej JPA vygenerovalo databázu podľa vášho návrhu. Implementujte tiež vhodný program na testovanie.

4 Obojsmerné asociácie

4.1 Obojsmerná 1:N - N:1 asociácia

Príklad 9: Knihy a ich vydavatelia

Príklad ilustruje obojsmernú verziu asociácií z príkladov 7 a 8

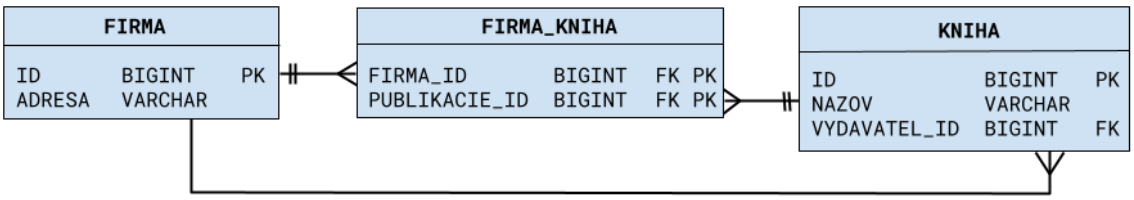


Entitné triedy - 1. verzia bez anotácií

```
@Entity
 public class Kniha implements Serializable {
   @Id
   private Long id;
   private String nazov;
   private Firma vydavatel;
```

```
@Entity
  public class Firma implements Serializable {
    @Id
    private Long id;
    private String adresa;
    private List<Kniha> publikacia;
```

Relačný diagram



Ako vidíme na diagrame, JPA interpretovalo referencie v triedach ako **dve nezávislé jednosmerné asociácie**. Obojsmerná asociácia je však tvorená dvoma jednosmernými, ktoré sú navzájom inverzné.

Entitné triedy - 2. verzia

Informáciu, že jedna referencia je inverzná k druhej, musíme povedať pomocou anotácie `@OneToMany(mappedBy="vydavatel")`.

```
@Entity
  public class Kniha implements Serializable {

    @ManyToOne
    private Firma vydavatel;
```

```
  @Entity
  public class Firma implements Serializable {

//  @JoinColumn(name = "VYDAVATEL_FK")
    @OneToMany(mappedBy="vydavatel")
    private List<Kniha> publikacia;
```

Anotácia `@OneToMany(mappedBy)` nahrádza `@JoinColumn(name)` používanú v prípade jednosmerných asociácií, pozri 2. verziu príkladu 8.

Pripomeňme, že pri 1:N aj N:1 asociáciách je cudzí kľúč vždy v tabuľke entity na viacnásobnej strane asociácie.

Dátový člen, ktorý je na cudzí kľúč namapovaný, sa označuje ako **vlastník asociácie**.
V našom príklade je to `vydavatel` v triede `Kniha`.

Zhrnutie Parameter `mappedBy` prepája dátové členy na oboch stranách asociácie:

- Používa sa len pri obojsmerných asociáciách a to na strane, ktorá nie je jej vlastníkom.
- Ukazuje na dátový člen referencovanej triedy, ktorý je vlastníkom asociácie, t.j. predstavuje inverznú referenciu.

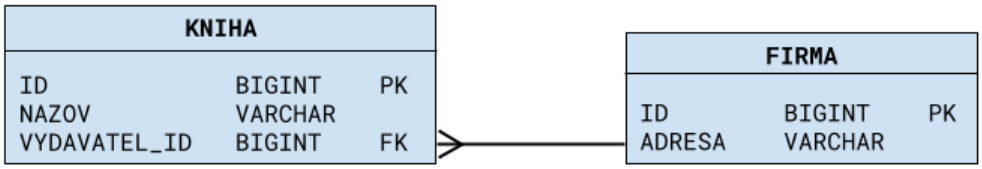
Poznámka:
Všimnite si, že `mappedBy` udáva meno dátového člena triedy, na rozdiel od anotácie `@JoinColumn`, ktorá udáva meno stĺpca.

Aplikácia
Aplikácia pomocou funkcie `create()` vytvorí vydavateľa s niekoľkými knihami a vloží ich do databázy.

```
Firma f = new Firma();
f.setAdresa("Veda");
Kniha k1 = new Kniha();
k1.setNazov("1984");
Kniha k2 = new Kniha();
k2.setNazov("Animal Farm");
k1.setVydavatel(f);
k2.setVydavatel(f);
// pre vytvorenie FK v db nie je nutne
List<Kniha> kl = new ArrayList<>();
kl.add(k1);
kl.add(k2);
f.setPublikacie(kl);
```

Poznámka: Pri vytváraní asociácie sme zadali referencie na oboch stranách asociácie. Pre správne vytvorenie hodnoty cudzieho kľúča v databáze však stačí zadať len stranu vlastníka t.j. `setVydavatel`. Pozri **cvičenie 4.1**.

Relačný diagram
Štruktúra databázy je rovnaká ako v prípade jednosmernej N:1 asociácie.

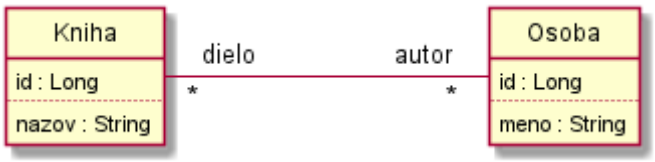


Implementáciu príkladu 9 nájdete v projekte **p41biN1** na disku STU.

4.2 Obojsmerná N:N asociácia

Príklad 10: Knihy a ich autori

Každá kniha pozná svojich autorov a každá osoba pozná svoje diela - knihy, ktorých je autorom.



Entitné triedy
Podobne ako v predchádzajúcom prípade, je potrebné povedať, že referencie `dielo` a `autor` sú navzájom inverzné. Keďže teraz sú obe referencie viacnásobné, vlastníkom môže byť hociktorá z nich. Na opačnej strane použijeme anotáciu `@ManyToMany(mappedBy)`.

```
@Entity
 public class Kniha implements Serializable {
   @Id
   private Long id;
   private String nazov;
   @ManyToMany
   private List<Osoba> autor;
```

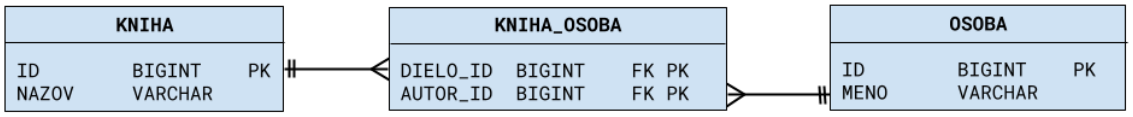
```
@Entity
 public class Osoba implements Serializable {
   @Id
   private Long id;
   private String meno;
   @ManyToMany(mappedBy="autor")
   private List<Kniha> dielo;
```

Aplikácia

Vytvorí a vloží do databázy niekoľko kníh s viacerými autormi, pričom niektoré knihy budú mať spoločných autorov. Viď UML diagram objektov v podkapitole Čítanie v prípade N:N asociácie.

Relačný diagram

Štruktúra databázy je rovnaká ako v prípade jednosmernej N:N asociácie.



Poznámka: Všimnite si však, že teraz sú názvy oboch stĺpcov prepojovacej tabuľky odvodené od názvov dátových členov - zoznamov referencií.

Implementáciu príkladu 10 nájdete v projekte **p42biNN** na disku STU.

4.3 Stratégia dotazovania

Pokračujeme v projektoch **p41biN1** a **p42biNN** z predchádzajúcich podkapitol.

Do súboru persistence.xml pridáme nasledujúce riadky, čím sa aktivuje logovanie SQL príkazov.

```
<property name="eclipselink.logging.level.sql" value="FINE"/>
<property name="eclipselink.logging.parameters" value="true"/>
```

Aplikácie doplníme o funkcie na čítanie údajov z databázy a po ich spustení v logu pozorujeme SQL dotazy generované JPA.

Čítanie v prípade N:1 asociácie

Do projektu **p41biN1** z príkladu **9** doplníme funkciu, ktorá nájde knihu podľa kľúča a načíta všetky informácie o nej.

```
public static void readKniha(long id) {
  EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");
  EntityManager em = emf.createEntityManager();
  Kniha k = em.find(Kniha.class, id);
  System.out.println("Nazov knihy: " + k.getNazov());
  System.out.println("Adresa vydavatela: " + k.getVydavatel().getAdresa());
}
```

V logu si môžeme všimnúť, že metóda `find` načíta všetky údaje o hľadanej knihe naraz, vrátane údajov o jej vydavateľovi. (Vyšle dva dotazy: pre dotazovanú entitu `KNIHA` aj referencovanú entitu `FIRMA`, výpisy názvu knihy a adresy vydavateľa nasledujú až za nimi.)

```
SELECT ID, NAZOV, VYDAVATEL_ID FROM KNIHA WHERE (ID = ?)
        bind => [201]
SELECT ID, ADRESA FROM FIRMA WHERE (ID = ?)
        bind => [101]
Nazov knihy: 1984
Adresa vydavatela: Veda
```

Čítanie v prípade 1:N asociácie

Do toho istého projektu doplníme funkciu, ktorá nájde firmu podľa kľúča a načíta všetky informácie o nej.

```
public static void readFirma(long id) {
  EntityManagerFactory emf = Persistence.createEntityManagerFactory("vsaPU");
  EntityManager em = emf.createEntityManager();
  Firma f = em.find(Firma.class, id);
  System.out.println("Adresa firmy: " + f.getAdresa());
  System.out.println("Pocet publikacii: " + f.getPublikacie().size());
}
```

V logu si môžeme všimnúť, že metóda `find` vyšle teraz `SELECT` len pre dotazovanú entitu `Firma`. Dotaz na asociovanú entitu t.j. tabuľku `KNIHA` vyšle až neskôr volanie metódy `getPublikacie()`.

```
SELECT ID, ADRESA FROM FIRMA WHERE (ID = ?)
        bind => [101]
Adresa firmy: Veda
SELECT ID, NAZOV, VYDAVATEL_ID FROM KNIHA WHERE (VYDAVATEL_ID = ?)
        bind => [101]
Pocet publikacii: 2
```

Zhrnutie - Stratégia dotazovania (FetchType)

V prvom prípade, keď išlo o asociáciu N:1, sa s dotazovaným objektom načítali údaje aj s asociovaným objektom. Táto stratégia - FetchType sa nazýva **EAGER**.

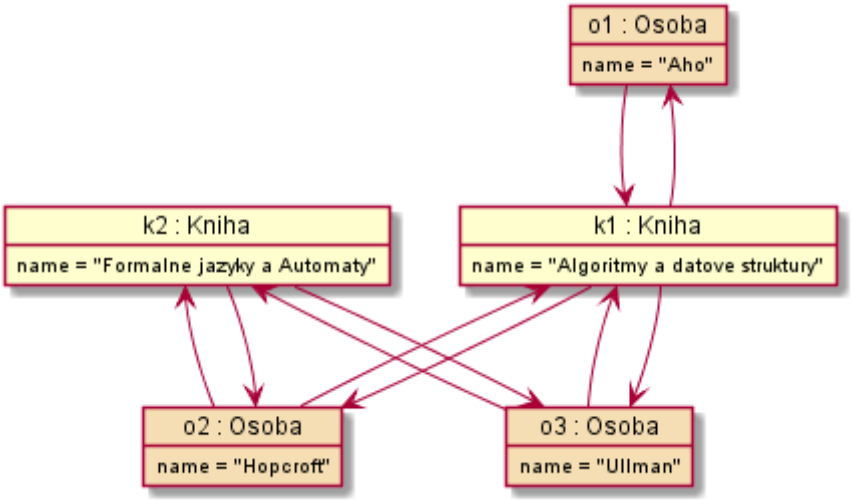
V druhom prípade, keď išlo o asociáciu 1:N, sa údaje o asociovaných objektoch nenačítali hneď, ale až keď boli potrebné. Táto stratégia - FetchType sa nazýva **LAZY**.

Uvedené správanie môžeme zmeniť v anotáciách parametrom `fetch`, napríklad:
`@OneToMany(fetch=FetchType.EAGER)` alebo `@ManyToOne(fetch=FetchType.LAZY)`

Čítanie v prípade N:N asociácie

Nastavenie EAGER v prípade viacnásobných referencií môže spôsobiť zbytočné záťažové problémy.

Knihy, osoby a ich vzájomé referencie vytvorené aplikáciou v projekte **p42biNN** (z príkladu **10**) zobrazuje UML diagram objektov:



Pre zoznamy referencií na oboch stranách asociácie nastavme FetchType.EAGER.

```
    @ManyToMany(fetch = FetchType.EAGER)
    private List<Osoba> autor;
```

```
    @ManyToMany(mappedBy="autor", fetch = FetchType.EAGER)
    private List<Kniha> dielo;
```

Keď teraz v aplikácii vyhľadáme podľa kľúča jedinú osobu, uvidíme v logu, že sa tým spustila celá lavína SQL dotazov, ktorými sa rekurzívne čítajú aj ďalšie knihy a osoby:

```
SELECT ID, MENO FROM OSOBA WHERE (ID = ?)
        bind => [333]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?)
    AND (t1.ID = t0.dielo_ID))
        bind => [333]
```

```
SELECT t1.ID, t1.MENO FROM KNIHA_OSOBA t0, OSOBA t1 WHERE ((t0.dielo_ID = ?)
    AND (t1.ID = t0.autor_ID))
        bind => [52]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?)
    AND (t1.ID = t0.dielo_ID))
        bind => [53]
SELECT t1.ID, t1.MENO FROM KNIHA_OSOBA t0, OSOBA t1 WHERE ((t0.dielo_ID = ?)
    AND (t1.ID = t0.autor_ID))
        bind => [51]
SELECT t1.ID, t1.NAZOV FROM KNIHA_OSOBA t0, KNIHA t1 WHERE ((t0.autor_ID = ?)
    AND (t1.ID = t0.dielo_ID))
        bind => [54]
```

Načítali sa všetky objekty, ku ktorým vedie od dotazovanej osoby cesta v grafe referencií.

Cvičenia

V nasledujúcich úlohách vytvárame rôzne modifikácie projektov z prednášky.

Cvičenie 4.1 Modifikácie projektu **p41biN1**

Všimnite si, že vo funkcii `create` sú zadané referencie na oboch stranách asociácie: Knihám sme zadali referenciu na firmu, ktorá ich vydáva, a firme sme vložili vydávané knihy do zoznamu diel.

Upravte funkciu `create` tak, že:

a) len knihám zadáte referenciu na vydavateľa. Po jej zavolaní sa presvedčte, že v databáze je FK stĺpec správne vyplnený. Následne použite funkciu readFirma na načítanie údajov z databázy a presvedčte sa, že v zozname diel sú už referencie na vydávané knihy.

b) len firme vložíte vydávané knihy do zoznamu diel (knihám však vydavateľa nezadáte).

Presvedčte sa, či po zavolaní funkcie bude v databáze cudzí kľúč správne vyplnený.

Cvičenie 4.2 Modifikácie projektu **p42biNN**

Upravte funkciu `create` tak, že vytvoríte dve knihy a ich autorov: Brave new world, ktorej autorom je Huxley a 1984 od Orwella. Osobám priradíte ich diela správne, ale pri zadávaní autorov kníh ich vymeníte (ako autora Brave new world uvediete Orwella, autorom 1984 bude Huxley).

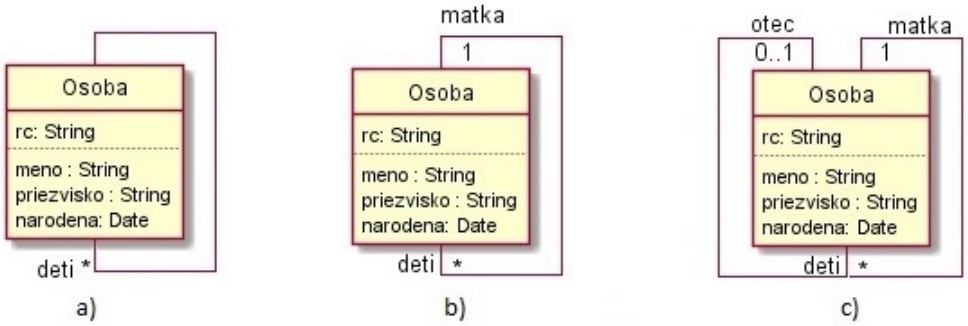
a) Po zápise do databázy sa v nej (v prepojovacej tabuľke) presvedčte, či sú diela priradené autorom správne alebo naopak.

b) skúste zmeniť anotácie `ManyToMany` tak, aby dielo bolo vlastníkom asociácie. Vytvorte databázu znovu a presvedčte sa, že teraz sú v nej diela priradené správne.

Vysvetlenie: Pri obojstrannej asociácii je pre zápis do databázy určujúca referencia na strane vlastníka.

Cvičenie 4.3

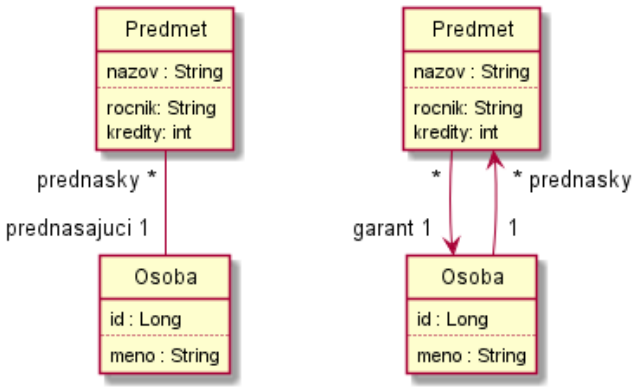
Skúste reimplementovať cvičenie 3.6, tak, že asociácie medzi deťmi a rodičmi budú obojsmerné.



Poznámky: Vo verzii a) diagramu nie je zadaný názov pre opačnú stranu asociácie. Názov dátového člena pre opačnú stranu asociácie si môžete vhodne zvoliť sami.

Pokiaľ budete mať problém, nájsť uspokojivé riešenie pre verziu c) UML diagramu, zvážte úpravu modelu tak, aby vyjadrovala skutočnosť, že otcom musí byť muž a matkou žena. Pozri cvičenie 6.6.

Cvičenie 4.4 UML diagrami na obrázku ilustrujú rozdiel medzi obojsmernou asociáciou a dvoma jednosmernými asociáciami.

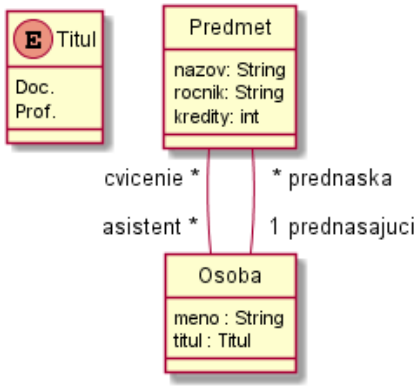


(a) Obojsmerná asociácia (b) Dve jednosmerné asociácie

Úlohy:

- Pre oba UML diagramy si najprv nakreslite, ako bude vyzerať štruktúra zodpovedajúcej databázy.
- Potom pre každý diagram vytvorte projekt, implementujte v ňom entitné triedy podľa diagramu a jednoduchý testovací program, ktorý vytvorí nejaké predmety, osoby a asociácie medzi nimi.
- V prípade (b) si overte, že garant predmetu nemusí predmet prednášať, t.j. nemusí ho mať medzi svojimi prednáškami. Prednášajúci môže byť iná osoba.
- Po spustení programov si overte, či vygenerované databázy majú takú štruktúru, ako ste očakávali.

Cvičenie 4.5 Pre UML diagram tried znázornený na obrázku, navrhnite štruktúru databázy a nakreslite si jej relačný diagram. Poznámka: Symbol (E) označuje enumeračný typ.



Vytvorte nový projekt a v ňom implementujte:

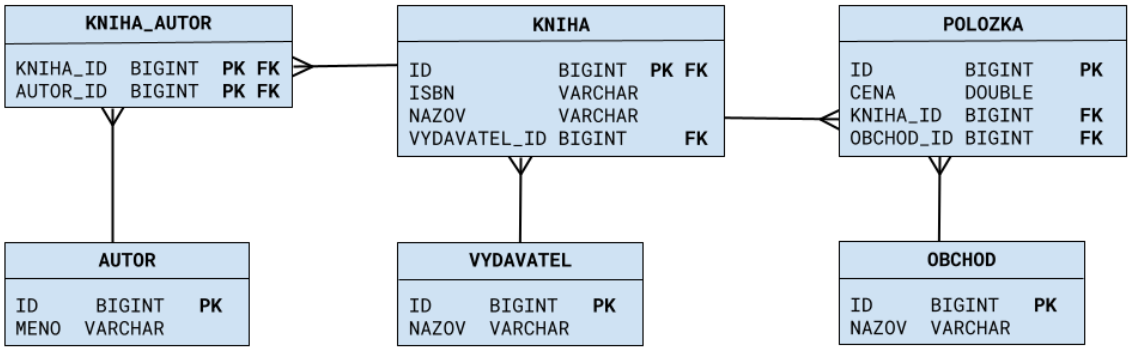
a) Entitné triedy, tak aby odpovedali vášmu návrhu DB (kľúčové atribúty môžu byť autogenerované) pričom asociácie sú obojsmerné a majú kardinalitu tak, ako je to na diagrame. Implementujte funkciu create, ktorá vytvorí dva predmety a tri osoby, pričom:
   1. osoba bude prednášať 1. predmet,
   2. osoba bude prednášať aj cvičiť 2. predmet
   3. osoba bude cvičiť 1. aj 2. predmet

   Jej spustením vygenerujete databázu. Overte, či má štruktúru akú ste navrhli.

b) Funkciu `void read(String meno)`, ktorá dostane meno osoby, nájde všetky predmety, ktoré cvičí (ako asistent). Na obrazovku vypíše jeho názov a meno prednášajúceho.

c) Funkciu `void release(Osoba o)`, ktorá dostane ako parameter osobu a zistí, či je taká osoba v DB. Ak áno, odstráni ju, pričom odstráni aj všetky odkazy na ňu, t.j. nesmie ostať už ani prednášajúcim ani cvičiacim/asistentom žiadneho predmetu.

Cvičenie 4.6 Na obrázku je ER diagram zobrazujúci štruktúru databázy portálu o knihách.



a) Navrhnite entitné triedy odpovedajúce ER diagramu, (t.j. JPA by z entitných tried mal automaticky vygenerovať tabuľky tak, ako sú zobrazené na diagrame):
   - Načrtnite si najprv UML diagram tried.
   - Pri návrhu asociácií zoberte do úvahy, akým spôsobom budete chcieť navigovať medzi objektami entitných tried.

b) Implementujte funkciu, ktorá vyhľadá knihu podľa ISBN.

c) Implementujte funkciu, ktorá vyhľadá autora podľa mena.

d) Implementujte funkciu, ktorá načíta z databázy všetky obchody a pre každý obchod vypíše všetky údaje o knihách (aj autoroch) v ponuke.

e) Implementujte funkciu, ktorá dostane ako argument objekt triedy Autor a vráti zoznam všetkých obchodov (objektov triedy Obchod), ktoré ponúkajú knihu od daného autora (t.j. je jedným z jej autorov). Funkcia môže predpokladať, že objekt, ktorý dostane ako argument, je manažovaný, t.j. bol načítaný z databázy.

f) Implementujte program, ktorý otestuje funkčnosť oboch funkcií.
Pre naplnenie databázy testovacími údajmi implementujte funkciu, ktorá importuje do DB dáta zo súboru **data4cv45.csv** na disku STU.

Cvičenie 4.7 Modifikujte 'ER diagram' z predchádzajúceho cvičenia, tak aby zaručoval, že cena knihy bude vo všetkých obchodoch rovnaká. Prispôsobte tomu entitné triedy, reimplementujte všetky funkcie a program.

5 Špecifické situácie

V tejto kapitole sa budeme venovať mapovaniu v niektorých menej bežných situáciách:

- kompozícia,
- 1:1 asociácia,
- mapovanie dvoch/viacerých tried do jednej tabuľky,
- mapovanie jednej triedy do viacerých tabuliek,
- zložené kľúče.

5.1 Kompozícia

Kompozícia je špeciálny prípad `OneToMany` asociácie, v ktorej **asociované objekty - komponenty môžu existovať len ako súčasť celku - kontajnera**.

Príklad 11: Nákupný košík

Nákupný košík obsahuje - vlastní viacero položiek s vybraným tovarom. Položky môžu existovať len ako súčasť košíka, ide teda o vzťah kompozície.

Asociácia medzi kontajnerom a komponentami sa zvyčajne realizuje ako jednosmerná: kontajner pozná spravované komponenty, ale komponenty nemusia poznať kontajner, v ktorom sú. Kardinalita je spravidla 1:N, nemusí byť teda v diagrame vyznačená.

Entitné triedy

Pre podporu implementácie špecifického správania kompozície poskytuje JPA ďalšie špeciálne argumenty anotácií.

```
@Entity
public class Polozka implements Serializable {
    @Id
    private Long id;
    private String tovar;
    private int mnozstvo;
```

```
@Entity
public class Kosik implements Serializable {
    @Id
    private Long id;
    @OneToMany(cascade=CascadeType.REMOVE, orphanRemoval=true)
    @JoinColumn(name="KOSIK_FK")
    private List<Polozka> polozky;
```

Parameter `cascade=CascadeType.REMOVE` zabezpečí, že pri odstránení košíka z databázy budú s ním automaticky z databázy odstránené aj všetky položky.

Parameter `orphanRemoval=true` zabezpečí, že ak sa položka odstráni z košíka (zo zoznamu položky) bude nielen vynulovaná FK hodnota v tabuľke `KOSIK`, ale odstránený aj záznam z tabuľky `POLOZKA`. (samotné `CascadeType.REMOVE` nestačí)

Implementáciu nákupného košíka nájdete v projekte **p51copmosit** na disku STU.

Projekt obsahuje tri funkcie:

- create - vytvorí košík s niekoľkými položkami. (Jej spustením môžeme vygenerovať aj tabuľky.)
- removeKosik - odstráni košík aj s položkami
- removePoložka - odstráni položku z košíka

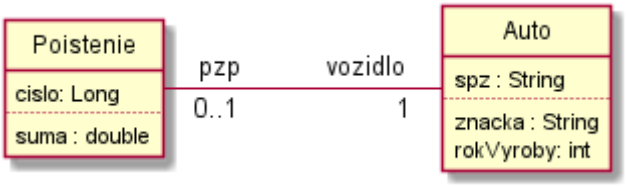
Relačný diagram

Štruktúra databázy je podobná ako v prípade `OneToMany` asociácie.

5.2 Asociácia 1:1

Príklad 12: Povinné zmluvné poistenie auta

Typickou ukážkou 1:1 asociácie medzi dvoma entitnými triedami je vzťah medzi vozidlom a jeho povinným poistením:



Entitné triedy

Pre 1:1 asociácie má JPA anotáciu `@OneToOne`. Takto anotovaný atribút bez ďalších JPA anotácií a parametrov je namapovaný na samostatný FK stĺpec, teda štruktúra databázy bude rovnaká ako pri asociácii `ManyToOne`.

```
@Entity
public class Poistenie implements Serializable {
    @Id
    private Long cislo;
    private double suma;
    @OneToOne
    @JoinColumn(nullable=false)    // vozidlo 1
    private Auto vozidlo;
```

```
@Entity
public class Auto implements Serializable {
    @Id
    private String spz;
    private String znacka;
    private int rokVyroby;
    @OneToOne(mappedBy="vozidlo")
    private Poistenie pzp;
```

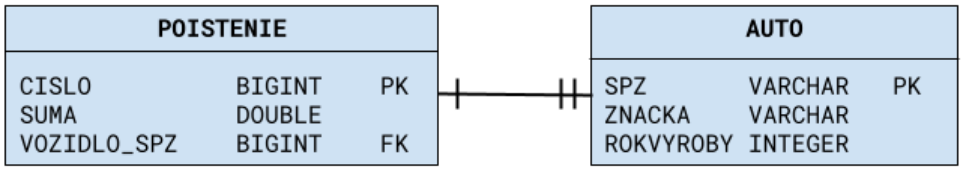
Poznámky:

- Parameter `nullable=false` v anotácii referencie na vozidlo zabezpečuje, že poistenie musí mať zadané vozidlo (v databáze), ako vyžaduje kardinalita 1 na strane vozidla v diagrame.
- Vlastník (owner) asociácie môže byť ktorákoľvek strana.
- Asociácie s kardinalitou 1:1 môžu byť tiež jednosmerné.

Relačný diagram

Skutočnosť, že ide o vzťah `OneToOne` vyjadruje v diagrame symbol | na strane poistenia (namiesto kuracích paprčiek).

Štruktúra databázy je však podobná ako pri `ManyToOne` asociácii, len FK stĺpec by mal mať definovaný `unique` constraint. Pozri cvičenie 5.3.

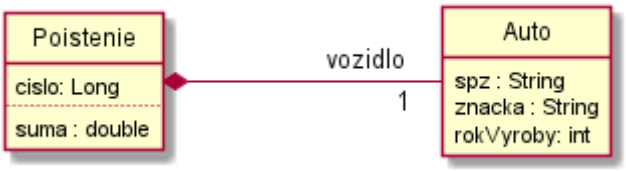


Implementáciu príkladu 12 nájdete v projekte **p52bi11** na disku STU.

5.3 Mapovanie dvoch tried do jednej tabuľky

Príklad 13: Povinné zmluvné poistenie auta - verzia 2.

Povinné zmluvné poistenie môžeme modelovať ako kompozíciu s kardinalitou 1:1. Poistenie bude kontajner a objekt obsahujúci informácie o poistenom aute je komponent tohto kontajnera.



Entitné triedy

Keďže informácie o poistenom aute sú súčasťou poistenia, môžeme ich atribúty v databáze ukladať priamo do tabuľky `POISTENIE`. V tomto prípade je len trieda `Poistenie` entitná, trieda `Auto` je vnorená.

- Pre anotáciu referencie na asociovaný objekt vnorenej triedy sa použije `@Embedded`.
- Pre anotáciu vnorenej triedy sa použije `@Embeddable` (pričom nemá PK atribút).

```
@Entity
public class Poistenie implements Serializable {
    @Id
    private Long cislo;
    private double suma;
//    @OneToOne
    @Embedded
    private Auto vozidlo;
```

```
@Embeddable
public class Auto implements Serializable {
    private String spz;
    private String znacka;
    private int rokVyroby;
```

Poznámka: Auto už nie je entitná trieda. Vložiť do databázy ho možno len ako súčasť poistenia, teda nie je pre neho ani potrebné volať persist.

Relačný diagram má len jednu tabuľku

POISTENIE		
CISLO	BIGINT	PK
SUMA	DOUBLE	
SPZ	VARCHAR	
ZNACKA	VARCHAR	
ROKVYROBY	INTEGER	

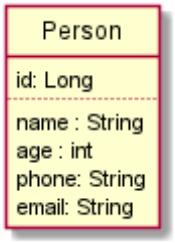
Implementáciu príkladu 13 nájdete v projekte **p53embed** na disku STU.

5.4 Mapovanie jednej triedy do viacerých tabuliek

Opačná situácia je, keď chceme atribúty entitnej triedy rozdeliť do viacerých tabuliek.

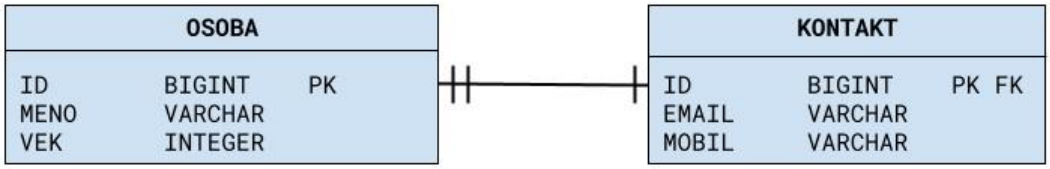
Príklad 14: Osoba a jej kontaktné údaje

V aplikácii chceme mať všetky informácie o osobe sústredené v jedinej entitnej triede `Person`.



V databáze sú však informácie o osobe rozdelené do dvoch tabuliek: Základné informácie sú v primárnej tabuľke `OSOBA`, doplňujúce - kontaktné údaje sa nachádzajú v samostatnej podriadenej tabuľke `KONTAKT`.

Relačný diagram



Všimnite si, že primárny kľúč podriadenej tabuľky je zároveň cudzím kľúčom do primárnej tabuľky entity. Tabuľky sú teda vo vzťahu 1:1.

Entitné triedy

Entitnú triedu v tomto prípade mapujeme na dve tabuľky, pričom podriadená tabuľka bude sekundárna.

```
@Entity
@Table(name = "OSOBA")
@SecondaryTable(name="KONTAKT")
public class Person implements Serializable {
    @Id
    private Long id;
    @Column(name="MENO")
    private String name;
    @Column(name="VEK")
    private int age;
    @Column(table= "KONTAKT")
    private String email;
    @Column(table= "KONTAKT",name="MOBIL")
    private String phone;
```

Názov sekundárnej tabuľky zadáme pomocou anotácie entitnej triedy `@SecondaryTable`. Meno tabuľky aj stĺpca, na ktorý je konkrétny dátový člen namapovaný, určíme pomocou parametrov anotácie `@Column`.

Poznámka: Sekundárnych tabuliek môže byť aj viac.

Implementáciu príkladu 14 nájdete v projekte **p54secondary** na disku STU.

5.5 Zložené kľúče

Tabuľky so zloženými kľúčmi sa uplatňujú a používajú v rôznych situáciách:

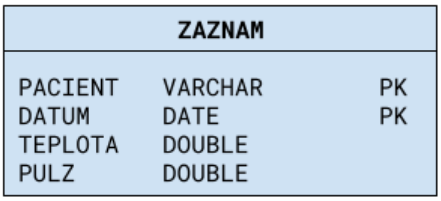
- Pri kompozícii, keď primárny kľúč komponentu je zložený z primárneho kľúča kontajnera a identifikátora komponentu v rámci kontajnera.
- Pri entitách, ktoré treba mať uložené vo viacerých verziách.
- V historizovaných entitách, ktoré si okrem aktuálneho stavu uchovávajú aj predchádzajúce.

Zložený kľúč má v takom prípade dve zložky: identifikátor entity a číslo verzie resp. údaj o čase.

Príklad 15: Zdravotné záznamy

Každodenné záznamy o teplote, tlaku ... pacienta počas jeho liečenia je vhodné ukladať v tabuľke so zloženým kľúčom.

Relačný diagram



Entitné triedy

Pre tabuľku so zloženým kľúčom, je potrebné okrem entitnej triedy vytvoriť aj ďalšiu triedu, ktorá obsahuje atribúty pre kľúčové stĺpce tabuľky. Trieda reprezentujúca kľúč je `@Embeddable`

```
@Embeddable
public class ZaznamPK implements Serializable {
    private pacient String;
    @Temporal(TemporalType.DATE)
    private Date datum;
```

V entitnej triede sa použije na označenie kľúča namiesto @Id anotácia `@EmbeddedId`

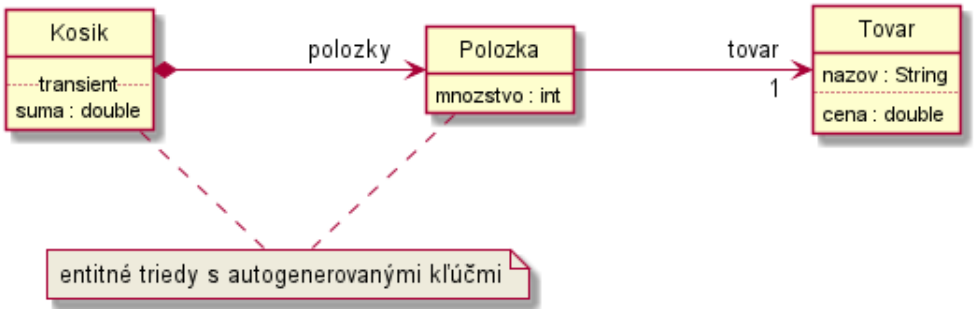
```
@Entity
public class Zaznam implements Serializable {
    @EmbeddedId
    private ZaznamPK pk;
    private double pulz;
    private double teplota;
```

Implementáciu príkladu 15 nájdete v projekte **p55compkey** na disku STU.

Cvičenia

Cvičenie 5.1 Reimplementujte projekt **p51composit** tak, aby asociácia medzi košíkom a položkami bola obojsmerná.

Cvičenie 5.2 Reimplementujte triedy nákupného košíka podľa UML diagramu:



Tovar bude samostatná entitná trieda.

Triedy `Kosik` a `Polozka` majú tabuľky s autogenerovanými kľúčmi (nie sú vyznačené v diagrame).

Košík má atribút `suma`, ktorý udáva aktuálnu celkovú cenu tovarov v košíku. Tento atribút sa nezapisuje do databázy (je transient)

Implementujte dve funkcie:

- `pridajTovar(Kosik k, Tovar t)` Pridá tovar do košíka: Ak tovar v košíku ešte nie je, pridá novú položku. Ak už v košíku tovar bol, zvýši jeho množstvo.
- `odoberTovar(Kosik k, Tovar t)` Odoberie tovar z košíka, t.j. zníži množstvo tovaru o 1. Ak klesne množstvo na 0, odstráni položku. Ak už košík nebude obsahovať žiadne položky, odstráni ho z databázy.

Cvičenie 5.3

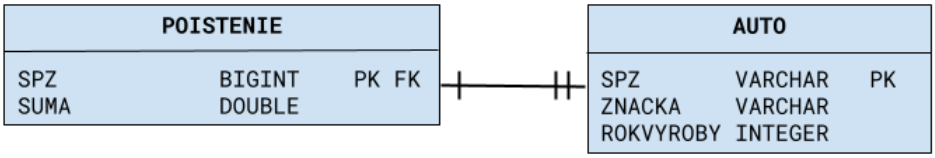
a) Skúste, či je do databázy vytvorenej v projekte **p52bi11** možné vložiť (manuálne alebo aplikáciou) viac poistení pre to isté auto.
Ak je to možné, zrejme JPA nevygeneroval v databáze `unique` constraint na FK stĺpec, aj napriek anotácii `OneToOne`. (Overiť si to môžete vo vlastnostiach indexu pre cudzí kľúč.)
Pridajte referenciu na vozidlo ešte anotáciu `JoinColumn(unique=true)` a pregenerujte databázu. Teraz by už malo vytvoriť unique-index pre FK, ktorý nedovolí vložiť viac poistení pre jedno auto.

b) Upravte projekt tak, aby bola asociácia jednosmerná: **Poistenie → Auto** (stačí odstrániť v aute referenciu pzp).

V aplikácii vytvorte dve zmluvy pre to isté auto a zistite, či sa ich podarí vložiť do databázy.

Cvičenie 5.4

1:1 asociáciu medzi poistením a autom by bolo možné namapovať na databázu tak, že SPZ by slúžila ako primárny kľúč, nielen v tabuľke AUTO, ale aj v tabuľke POISTENIE.



Návod: Pozri JPA Tutorial - JPA OneToOne Primary Key[11].

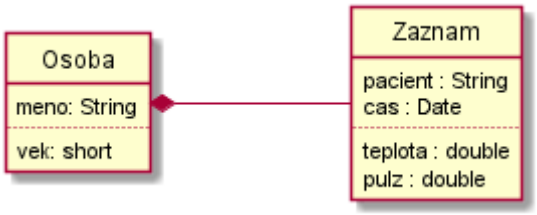
Cvičenie 5.5

V projekte **p53embed** implementujte funkciu `List<Poistenie> findByYear(int year)`, ktorá pomocou JPQL-dotazu vyhľadá všetky poistné zmluvy pre autá so zadaným rokom výroby.

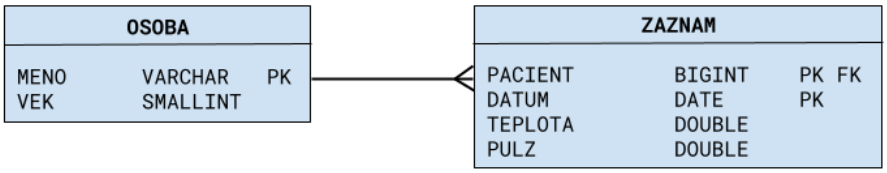
Cvičenie 5.6

Do projektu `p55compkey` doplňte entitnú triedu `Osoba` obsahujúcu osobné údaje pacientov. Vzťah medzi zdravotným záznamom a osobou modelujte ako kompozíciu podľa UML diagramu:

[11] http://www.java2s.com/Tutorials/Java/JPA/0730__JPA_OneToOne_Primary_Key_Join.htm



Pomocou JPA anotácií namapujte entitné triedy na databázu, ako je znázornená na diagrame. Tabuľka `ZAZNAM` má zložený kľúč a jeho prvá zložka je zároveň cudzí kľúč odkazujúci na primárny kľúč tabuľky `OSOBA`, teda meno osoby, ktorá je pacientom.



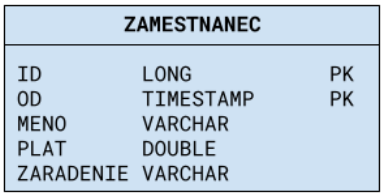
Návod: Ak nebudete vedieť, ako správne dať anotácie, aby sa vygenerovala databáza podľa diagramu, skúste opačný postup: Pridajte do databázy vytvorenej projektom **p55compkey** tabuľku `OSOBA` a tabuľke `ZAZNAM` pridajte foreign key `PACIENT → OSOBA.MENO`. V novom projekte vygenerujte z databázy entitné triedy a pozrite sa, aké anotácie sú v nich použité.

Cvičenie 5.7 Práca s historizovanou tabuľkou

Organizácia eviduje v databáze údaje o svojich zamestnancoch - meno, pracovné zaradenie, plat, a.t.ď. Viaceré z týchto údajov sa môžu časom meniť. Aby bolo možné zistiť nielen aktuálny stav (napr. plat) ale aj stav kedykoľvek v minulosti, sú údaje uložené v tzv. historizovanej tabuľke, v ktorej kľúč je zložený z dvoch stĺpcov:

- `ID LONG` identifikátor entity - zamestnanca
- `OD TIMESTAMP` čas zmeny, t.j. odkedy platia nové údaje

Ďalej tabuľka obsahuje historizované údaje o zamestnancovi, viď diagram:



Vytvorte entitnú triedu pre historizovanú tabuľku `ZAMESTNANEC`. Implementujte a otestujte funkcie pre prácu s informáciami o zamestnancoch:

- `vyhladatZamestnanca(long id)` Vráti aktuálne údaje o zamestnancovi.
- `vyhladatZamestnanca(long id, Date d)` Vráti údaje o zamestnancovi platné k danému dátumu.
- `prijatZamestnanca(long id, String meno)` Vytvorí nového zamestnanca.
- `aktualizovatZamestnanca(Zamestnanec z)` Aktualizuje údaje o zamestnancovi.
- `prepustitZamestnanca(Zamestnanec z)` Prepustí zamestnanca.

Dôležité: V historizovanej tabuľke sa nikdy záznamy nevymazávajú ani nemodifikujú, len vkladajú. To značí, že:

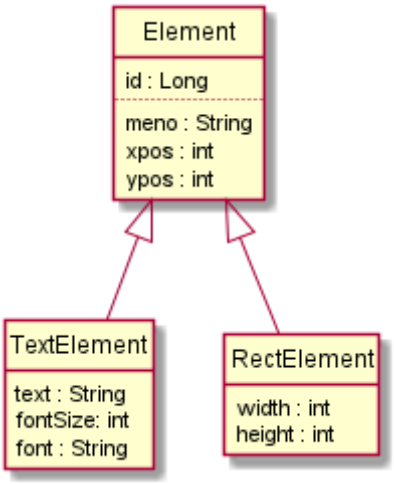
- Funkcia `aktualizovatZamestnanca` musí namiesto UPDATE volať INSERT, ktorý pridá nový záznam s aktualizovanými údajmi do tabuľky `ZAMESTNANEC`. Kľúčový stĺpec `OD` bude obsahovať timestamp okamihu zmeny (použite `new Date()`)
- Funkciu `prepustitZamestnanca` môžeme implementovať volaním funkcie pre aktualizáciu údajov `aktualizovatZamestnanca`, pričom hodnoty zo stĺpcov `MENO, PLAT` a `ZARADENIE` vymažeme.

6 Mapovanie hierarchie tried

Entitno relačné modelovanie nepozná pojem dedičnosti a relačné databázy nemajú štandardne podporu pre modelovanie hierarchie entít. Pri návrhu štruktúry databázy, ktorá by zachytila vzťahy dedičnosti medzi entitnými triedami, sa používa niekoľko návrhových vzorov - stratégií.

Ilustrujeme si ich na príklade hierarchie tried jednoduchého grafického rozhrania znázornenom na UML diagrame:

Príklad 16: Triedy grafického rozhrania



Entitné triedy

Všetky triedy hierarchie majú ten istý kľúčový dátový člen. Je definovaný v koreňovej triede hierarchie. Odvodené triedy ho dedia.

```
@Entity
public class Element implements Serializable {
    @Id
    private Long id;
    private String name;
    private int xpos ;
    private int ypos ;
```

```
@Entity
public class TextElement extends Element implements Serializable {
    private String text;
    private int fontSize;
    private String font;
```

```
@Entity
public class RectElement extends Element implements Serializable {
    private int width;
    private int height;
```

6.1 Stratégie mapovania

JPA poskytuje tri základné stratégie pre mapovanie hierarchie entitných tried do relačnej databázy:

- `SINGLE_TABLE` - jedna tabuľka
- `JOINED` - prepojené tabuľky
- `TABLE_PER_CLASS` - samostatné tabuľky

Stratégiu mapovania definujeme anotáciou `@Inheritance` koreňovej triedy, napr.

```
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public class Element implements Serializable {
```

Jedna tabuľka

Všetky dátové členy, všetkých tried hierarchie budú mapované do jednej spoločnej tabuľky.

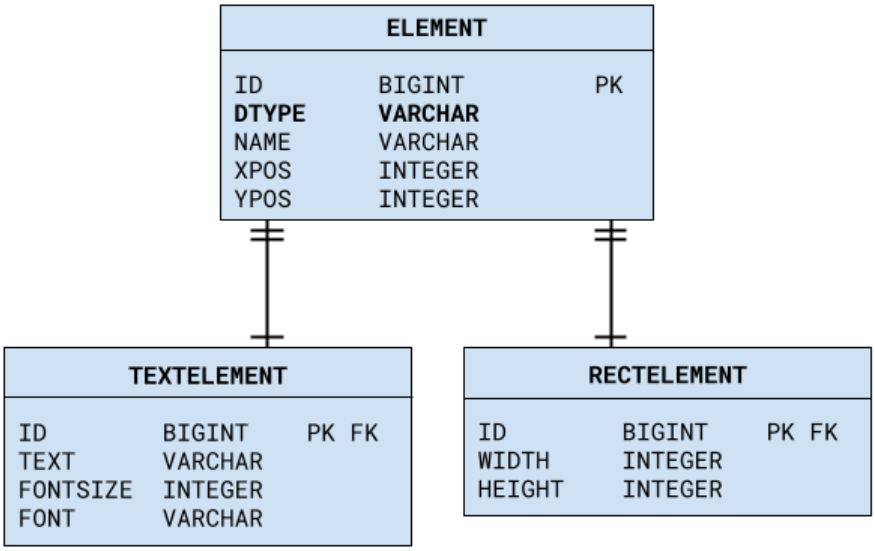
ELEMENT		
ID	BIGINT	PK
DTYPE	**VARCHAR**	
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
TEXT	VARCHAR	
FONTSIZE	INTEGER	
FONT	VARCHAR	
WIDTH	INTEGER	
HEIGHT	INTEGER	

Okrem stĺpcov odpovedajúcich dátovým členom obsahuje tabuľka stĺpec **`DTYPE`**, ktorý obsahuje názov skutočnej triedy objektu.

Poznámka: Táto stratégia je základná (default), anotácia `@Inheritance` nie je potrebná.

Prepojené tabuľky

Každá entitná trieda hierarchie má vlastnú tabuľku. Všetky tabuľky majú primárny kľúč, ktorý je mapovaný na kľúčový dátový člen (definovaný v koreňovej triede). Primárny kľúč je zároveň cudzím kľúčom do tabuľky koreňovej triedy. Okrem primárneho kľúča obsahujú len stĺpce pre dátové členy definované priamo v danej triede (nie zdedené)



Stĺpec **`DTYPE`**, ktorý obsahuje názov triedy objektu, sa nachádza len v tabuľke koreňovej triedy.

Samostatné tabuľky

Každá entitná trieda hierarchie má svoju vlastnú tabuľku, ktorá obsahuje stĺpce pre všetky dátové členy, vlastné aj zdedené:

ELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	

TEXTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
TEXT	VARCHAR	
FONTSIZE	INTEGER	
FONT	VARCHAR	

RECTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
WIDTH	INTEGER	
HEIGHT	INTEGER	

Medzi tabuľkami nie sú žiadne väzby. Trieda entitného objektu je určená priamo tabuľkou.

6.2 Mapovanie abstraktnej rodičovskej triedy

Ak je niektorá trieda v hierarchii abstraktná trieda, nie je možné vytvárať jej inštancie a teda nie je ani potrebné vytvárať jej tabuľku. V prípade stratégie `TABLE_PER_CLASS` sa ani nevytvorí. V prípade stratégie `JOINED` sa môžeme rozhodnúť, či vytvoríme tabuľku aj pre abstraktnú triedu alebo jej atribúty namapujeme do tabuliek jej podtried. V takom prípade použijeme namiesto `@Entity` anotáciu `@MappedSuperclass`.

```
@MappedSuperclass
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Element implements Serializable {
    @Id
    private Long id;
    private String name;
    private int xpos ;
    private int ypos ;
```

Poznámka: v tomto príklade je abstraktný koreň hierarchie, ale mohla by to byť aj iná trieda.

Relačný diagram

TEXTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
TEXT	VARCHAR	
FONTSIZE	INTEGER	
FONT	VARCHAR	

RECTELEMENT		
ID	BIGINT	PK
NAME	VARCHAR	
XPOS	INTEGER	
YPOS	INTEGER	
WIDTH	INTEGER	
HEIGHT	INTEGER	

Atribúty abstraktnej triedy sa presunuli do tabuliek podtried.

6.3 Výber stratégie

Z hľadiska použitia je asi najjednoduchšia stratégia SINGLE_TABLE. Nevýhoda je, že v prípade veľkého počtu atribútov špecifických pre podtriedy, vznikne tabuľka s veľkým počtom stĺpcov, ktoré sú však zväčša prázdne. Pre rozhodovanie medzi stratégiami JOINED a TABLE_PER_CLASS je rozhodujúci charakter dotazov, aké budú prevažovať pri vyhľadávaní a čítaní údajov z databázy.

Ilustrujú to nasledujúce jednoduché volania metód entity managera v príklade 16.

1. Vyhľadávanie podľa kľúča, ak triedu objektu poznáme

```
  TextElement e = em.find(TextElement.class, id);
  System.out.println(e.getText());
```

V prípade stratégie TABLE_PER_CLASS metóda `find` vygeneruje jednoduchý SQL dotaz, ktorý môžeme vidieť v logu:

```
--SELECT ID, FONT, FONTSIZE, NAME, TEXT, XPOS, YPOS FROM TEXTELEMENT WHERE (ID = ?)
```

V prípade stratégie JOINED je potrebné prepojenie niekoľkých tabuliek:

```
--SELECT t0.ID, t0.DTYPE, t0.NAME, t0.XPOS, t0.YPOS, t1.ID, t1.FONT, t1.FONTSIZE, t1.
    TEXT FROM ELEMENT t0, TEXTELEMENT t1 WHERE ((t0.ID = ?) AND ((t1.ID = t0.ID) AND (t0
    .DTYPE = 'TextElement')))
```

2. Vyhľadávanie podľa kľúča, keď triedu objektu nepoznáme

```
        Element e = em.find(Element.class, id);
        System.out.println(e);
        // skusme ci je to TextElement
        if (e instanceof TextElement) {
            System.out.println("" + ((TextElement) e).getText());
        }
```

V prípade stratégie TABLE_PER_CLASS musí manažér entít postupne dotazovať viaceré tabuľky, keďže nepoznáme konkrétnu triedu objektu a teda nevieme, v ktorej tabuľke sa záznam nájde:

```
--SELECT ID, HIGHT, NAME, WIDTH, XPOS, YPOS FROM RECTELEMENT WHERE (ID = ?)
--SELECT ID, FONT, FONTSIZE, NAME, TEXT, XPOS, YPOS FROM TEXTELEMENT WHERE (ID = ?)
```

V prípade stratégie JOINED si najprv samostatným dotazom zistí dátový typ.

```
--SELECT DISTINCT DTYPE FROM ELEMENT WHERE (ID = ?)
--SELECT t0.ID, t0.DTYPE, t0.NAME, t0.XPOS, t0.YPOS, t1.ID, t1.FONT, t1.FONTSIZE, t1.
    TEXT FROM ELEMENT t0, TEXTELEMENT t1 WHERE ((t0.ID = ?) AND ((t1.ID = t0.ID) AND (t0
    .DTYPE = 'TextElement')))
```

Pri všetkých troch stratégiách si dokáže manažér entít zistiť skutočnú triedu dotazovanej entity a načítať hodnoty všetkých dátových členov.

3. Načítanie vybraných údajov zo všetkých objektov

Napríklad načítanie súradníc všetkých elementov môžeme urobiť priamo pomocou SQL dotazu. V prípade stratégie JOINED stačí jednoduchý dotaz:

```
SELECT XPOS, YPOS FROM ELEMENT
```

V prípade stratégie TABLE_PER_CLASS je však potrebný UNION viacerých dotazov.

```
SELECT XPOS, YPOS FROM ELEMENT
UNION (SELECT XPOS, YPOS FROM TEXTELEMENT)
UNION (SELECT XPOS, YPOS FROM RECTELEMENT)
```

4. Načítanie všetkých údajov zo všetkých objektov

Načítať všetky údaje môžeme jednoduchým JPQL dotazom, rovnakým pre všetky stratégie:

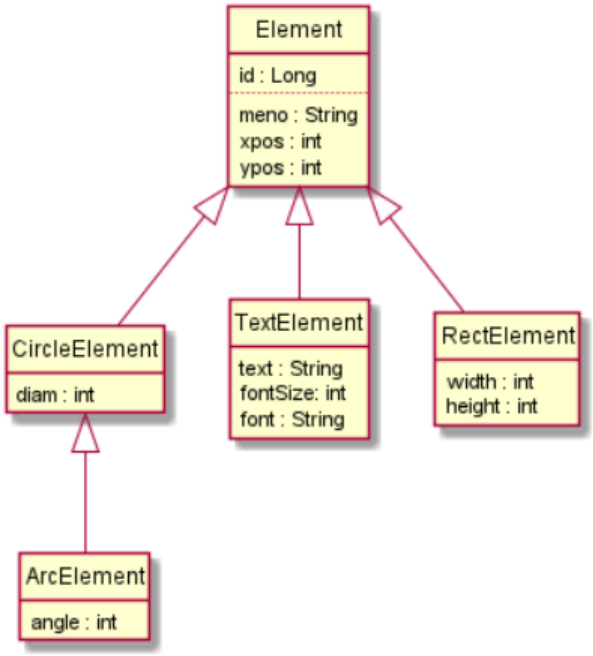
```
  Query q = em.createQuery("select e from Element e");
  List<Element> elements = q.getResultList();
  for (Element e : elements) {
    System.out.println(e.getName()+ " " + e.getClass().toString());
  }
```

Implementáciu príkladu 16 nájdete v projekte **p61** na disku STU.

Cvičenia

Cvičenie 6.1

V projekte **p61** z príkladu 16 doplňte hierarchiu tried o ďalšiu úroveň podľa UML diagramu.



Cvičenie 6.2

Pre každú zo stratégií mapovania tried z projektu **p61** napíšte SQL dotazy pre načítanie dát resp. objektov:

- počtu všetkých elementov (včítane podtried),
- počtu všetkých textových elementov,
- mien všetkých textových elementov,
- všetkých textových elementov,
- všetkých elementov s xpos > 0 (včítane podtried),
- všetkých textových elementov s xpos > 0.

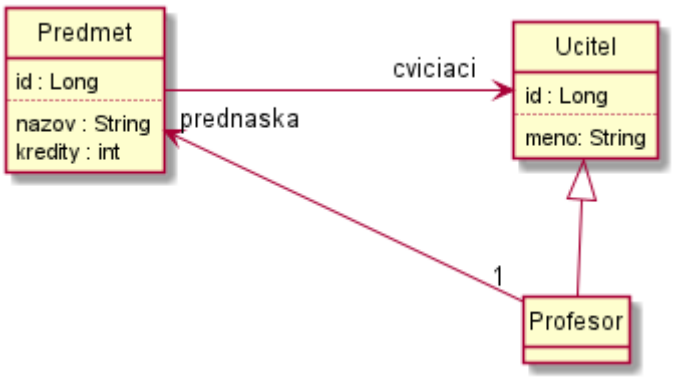
Cvičenie 6.3

Pre SQL-dotazy z predchádzajúceho cvičenia skúste nájsť vhodné JPQL alternatívy. Overte funkčnosť dotazov pre jednotlivé stratégie a v logu pozrite vygenerované SQL-dotazy.

Pozri dokumentáciu napríklad: Jakarta Persistence Query Language tutorial[12]

Cvičenie 6.4

a) Navrhnite a nakreslite štruktúru databázy pre entitné triedy znázornené v UML diagrame:



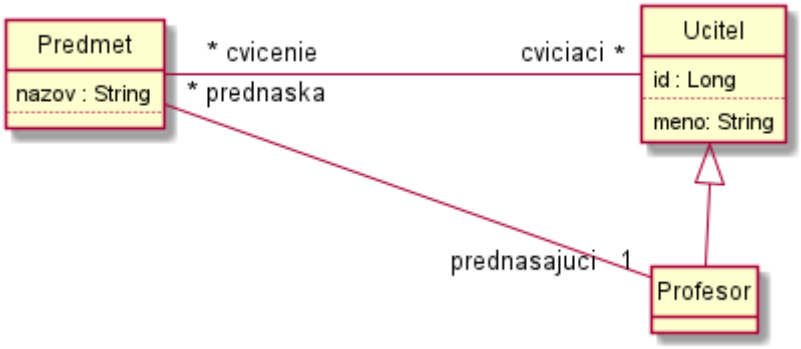
Urobte verzie návrhu pre každú zo stratégií mapovania hierarchie tried.

[12] https://jakarta.ee/learn/docs/jakartaee-tutorial/current/persist/persistence-querylanguage/persistence-querylanguage.html

b) Implementujte entitné triedy podľa UML diagramu a anotujte ich tak, aby sa namapovali na databázu, ako ste ju navrhli v časti a).
c) Naplňte databázu dátami pre testovanie (aspoň jeden predmet s prednášajúcim a niekoľkými cvičiacimi). Implementujte aplikáciu, ktorá z databázy načíta všetky údaje o predmetoch, pustite ju a overte, či sú všetky dátové členy objektov a asociácie medzi nimi správne načítané a vytvorené.
d) Implementujte funkciu, ktorá vráti zoznam všetkých prednášajúcich, ktorí si svoj predmet sami cvičia (sú medzi cvičiacimi predmetu, ktorý sami prednášajú).

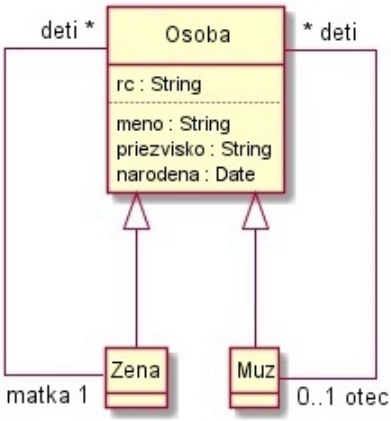
Cvičenie 6.5

Riešte predchádzajúcu úlohu pre verziu UML diagramu s obojsmernými asociáciami.



Cvičenie 6.6

Na obrázku je verzia UML diagramu cvičenia 3.6 resp. 4.3, ktorá zachytáva skutočnosť, že matka je žena a otec je muž.



Pre každú zo stratégií mapovania hierarchie najprv načrtnite, ako by vyzerala databáza odpovedajúca UML diagramu. Následne implementujte entitnú triedu podľa UML diagramu a použite JPA anotácie tak, aby z nej JPA vygenerovalo databázu podľa vášho návrhu. Implementujte tiež vhodný program na testovanie.

7 REST - Úvod

REST (Representational state transfer) je softvérový architektonický štýl navrhnutý pre vývoj architektúry pre World Wide Web. Kladie dôraz na škálovateľnosť interakcií medzi komponentmi, jednotné rozhrania, nezávislý vývoj a nasadenie komponentov.

Definuje súbor obmedzení (požiadaviek, zásad a princípov) kladených na architektúru zodpovedajúcu paradigme REST:

- **Klient-server** – klient a server sú nezávislé.
- **Bezstavový server** - nezaznamenáva stav klienta.
- **Využitie medzipamäte** – server označí dáta, ktoré možno ukladať do medzipamäte.
- **Jednotné rozhranie**
- **Viacvrstvový systém** – komponenty sprostredkujúce komunikáciu medzi klientom a serverom správanie neovplyvňujú.

7.1 REST - základné pojmy

Hoci REST bol pôvodne navrhnutý ako súbor princípov pre architektúru webu, má v súčasnosti oveľa širšie uplatnenie, najmä pri návrhu komunikačných rozhraní v distribuovaných aplikáciách a systémoch založených na webových službách.

Komunikačné rozhranie služby, navrhnuté na základe požiadaviek architektúry REST, sa označuje REST-API. V tomto kontexte je popri bezstavovej klient-server komunikácii kľúčovou požiadavka jednotného rozhrania.

Základné koncepty REST-API:

- **Zdroj** (resource) - **kľúčová abstrakcia**. Zdrojom môže byť akýkoľvek objekt nachádzajúci sa na webe, ktorý je jednoznačne identifikovaný svojím URI.
- **Klient-server komunikácia** - Klient komunikuje so serverom pomocou správ (požiadavky a odpovede), ktoré obsahujú reprezentáciu stavu zdroja. (Odtiaľ je odvodená skratka REST = representational state transfer.)
- **Reprezentácia** je dokument, ktorý zachytáva aktuálny alebo nový stav objektu. Typicky to je HTML, XML, JSON, prípadne jednoduchý textový dokument. Môže to však byť akýkoľvek súbor - obrázok, pdf, zip ...
- **Formát reprezentácie** v správe udáva **Internet Media Type** (skrátene MIME-typ).
- **Univerzálne rozhranie** - REST-API poskytuje pre manipuláciu so zdrojmi **jednotnú sadu CRUD operácií**.

RESTful web service

Webová služba založená na princípoch REST **poskytuje REST-API pre prístup ku kolekcii zdrojov**, ktoré spravuje. Zdroje typicky tvoria **hierarchickú štruktúru**, ktorú odzrkadľuje ich URL.

Príklad:

- Zdroj s URI `http://moja.skola.sk/resources/predmety` je koreň hierarchie zdrojov s informáciami o predmetoch,
- `http://moja.skola.sk/resources/predmety/VSA` je URI zdroja s informáciami k predmetu VSA,
- `http://moja.skola.sk/resources/predmety/VSA/P1` je URI dokumentu k 1. prednáške z VSA.

CRUD operácie pre manipuláciu so zdrojmi sú priamo namapované na požiadavky protokolu HTTP:

URI zdroja	GET	PUT	POST	DELETE
kolekcia zdrojov	Vráti vhodnú reprezentaciu zoznamu zdrojov, ktorá umožní prístup k jednotlivým zdrojom v kolekcii. Napr. zoznam relatívnych URI zdrojov v kolekcii.	Nahradí celú kolekciu novou. (Menej používané)	Pridá nový zdroj do kolekcie	odstráni celú kolekciu zdrojov
koncový zdroj (list v hierarchii)	Vráti reprezentáciu zdroja v požadovanom formáte (media type).	Aktualizuje stav zdroja resp. nahradí starý stav novým. Idempotentná metóda.	Považuje zdroj za kolekciu a vloží do nej nový pod-zdroj. (Menej používané)	odstráni zdroj

JAX-RS

Súčasťou platformy JakartaEE je knižnica JAX-RS implementujúca podporu pre vývoj RESTful web služieb v jazyku Java. Pozri napr. Jakarta JAX-RS tutorial[13] alebo Eclipse Jakarta EE tutorial[14].

Komponent poskytuje vývojárom predovšetkým sadu anotácií, umožňujúcich:

- Definovať URI zdrojov spravovaných REST službou.
- Prepojiť HTTP požiadavky s metódami zdroja, implementujúcimi CRUD operácie.
- Extrahovať údaje z HTTP požiadaviek a poskytnúť ich ako argumenty metódam zdroja.

REST službu je možné spustiť len ako webovú aplikáciu, teda nasadenú na webový server (kontajner), ktorý jej poskytuje infraštruktúru a rozhranie pre komunikáciu prostredníctvom protokolu HTTP. Webovú aplikáciu poskytujúcu REST službu budeme volať aj skrátene REST-aplikácia.

Poznámka: Existujú softvérové riešenia umožňujúce zabaliť do REST-aplikácie aj web kontajner (embedded web container). Potom sa dá REST služba spustiť ako obyčajná (standalone) aplikácia. Pri ukážkach v nasledujúcich kapitolách však vychádzame z toho, že REST-aplikácie sa budú nasadzovať na aplikačný server, konkrétne GlassFish verzia 8. Zoznam ďalších platforiem podporujúcich JakartaEE aplikácie nájdete na stránke Jakarta EE Compatible Produts[15].

7.2 Prvá REST aplikácia

REST služba vytvorená pomocou JAX-RS pozostáva z

- aplikačnej triedy, ktorá špecifikuje ostatné triedy implementujúce aplikáciu:
- jednej príp. viacerých tried zdroja
- a prípadne ďalších podporných tried.

Aplikačná trieda je odvodená od triedy `jakarta.ws.rs.core.Application;` pričom pomocou anotácie `@ApplicationPath` definuje relatívnu URI, ktorá slúži ako bázové URI všetkých spravovaných zdrojov.

```
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

@ApplicationPath("resources")
public class JakartaRestConfiguration extends Application { }
```

Trieda zdroja predstavuje prístupový bod ku zdroju, resp. kolekcii zdrojov. Trieda je označená anotáciou `@Path`, ktorá definuje relatívnu URI zdroja. Ak trieda sprístupňuje celú kolekciu zdrojov, je to URI koreňového zdroja v hierarchii zdrojov.

Trieda implementuje CRUD metódy na manipuláciu so zdrojmi. Tieto sú namapované na HTTP požiadavky pomocou anotácií `@GET, @PUT, @POST, @DELETE` ...

Metódy zdroja, ktoré posielajú obsah klientovi (GET, príp. POST), špecifikujú jeho formát (Internet media type) anotáciou `@Produces`.

[13] https://jakarta.ee/learn/starter-guides/how-to-build-a-restful-web-service/
[14] https://eclipse-ee4j.github.io/jakartaee-tutorial/
[15] https://jakarta.ee/compatibility/

Metódy zdroja, ktoré prijímajú obsah od klienta (PUT, POST) špecifikujú anotáciou `@Consumes`, aký formát obsahu akceptujú.

Príklad 17: REST služba s jedným zdrojom

Služba poskytuje prístup ku zdroju, ktorý predstavuje oznam vo forme jednoduchej textovej správy.

```
@Path("message")
public class SimpleResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getText() {
        return "Dobré ráno";
    }
}
```

V tejto najjednoduchšej verzii poskytuje trieda zdroja `SimpleResource` len metódu na spracovanie HTTP požiadavky GET, ktorá vráti klientovi oznam.

- Anotácia triedy `@Path("message")`, ktorá definuje relatívnu URI zdroja.
- Anotácia `@Produces(MediaType.TEXT_PLAIN)` určuje, že návratová hodnota metódy je jednoduchý text.

Po spustení aplikácie môžeme REST službu otestovať priamo prehliadačom odoslaním GET požiadavky pre URI zdroja: `<ip-servera>/<aplikacia>/resources/message`.

Triedu zdroja doplníme o metódu na spracovanie HTTP požiadavky PUT, ktorú môže klient použiť na zmenu textu oznamu.

```
@Path("message")
@ApplicationScoped
public class SimpleResource {
    private String sprava = "";

    @PUT
    @Consumes(MediaType.TEXT_PLAIN)
    public void putMessage(String content ) {
       sprava = content;
    }

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getMessage() {
        return sprava;
    }
}
```

Aktuálny stav zdroja uchovávame zatiaľ len v pamäti ako dátový člen triedy zdroja. Anotácia triedy zdroja `@ApplicationScoped` zabezpečí, že sa vytvorí len jedna inštancia triedy zdroja, ktorá spracuje všetky HTTP požiadavky na zdroj počas celého behu aplikácie.

Bez tejto anotácie by sa pre obslúženie každej požiadavky vytvárala vždy nová inštancia triedy zdroja. Pozri cvičenie 7.1.

Poznámka: Anotácia `@ApplicationScoped` sa nachádza v balíku `jakarta.enterprise.context`. Podobnú funkcionalitu je možné dosiahnuť aj inými anotáciami, napr. `@Stateles` z balíka `jakarta.ejb` alebo `@Singleton` z balíka `jakarta.inject`.

Implementáciu príkladu 17 nájdete v projekte **rs1simple** na disku STU.

REST klient

Klientom REST služby môže byť akákoľvek aplikácia, ktorá dokáže posielať HTTP požiadavky a prijímať odpovede. Knižnica JAX-RS má takisto podporu pre vytvorenie REST klienta v jazyku Java.

Inštancia triedy `jakarta.ws.rs.client.WebTarget` poskytuje metódy pre posielanie HTTP požiadaviek. Vytvoríme ju pomocou triedy `jakarta.ws.rs.client.ClientBuilder`, pričom definujeme URI zdroja. Následne ju môžeme použiť na prácu so zdrojom prostredníctvom HTTP požiadaviek.

```
String BASE_URI = "http://localhost:8080/rs1simple/resources";

Client client = ClientBuilder.newClient();
WebTarget webTarget = client.target(BASE_URI).path("message");

// putting message to rest service
String message = "Hello";
webTarget.request(MediaType.TEXT_PLAIN)
.put(Entity.entity(message, MediaType.TEXT_PLAIN));

// getting message from rest service
String result = webTarget.request(MediaType.TEXT_PLAIN).get(String.class);
System.out.println("GET returned: " + result);
```

Implementáciu klienta k príkladu 17 nájdete v projekte **rs1simpleClient** na disku STU.

7.3 REST služba s viacerými zdrojmi

Jedna REST služba môže poskytovať prístup k mnohým zdrojom prostredníctvom jednej alebo viacerých tried zdroja. Každá trieda môže implementovať prístupové metódy pre celú kolekciu zdrojov, ktoré sú hierarchicky usporiadané.

V nasledujúcom príklade ilustrujeme využitie základných JAX-RS anotácií pri implementácii REST služby mnohými zdrojmi.

Príklad 18: REST služba školy

Príklad ilustruje triedu zdroja, ktorá implementuje prístupové metódy pre kolekciu zdrojov reprezentujúcich informácie o škole a vyučovacích predmetoch.

```
@Path("skola")
public class SkolaResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getInfo() {
        return "FEI STU";
    }

    @GET
    @Path("adresa")
    @Produces(MediaType.TEXT_PLAIN)
    public String getAdresa() {
        return "Ilkovicova 2, Bratislava";
    }
```

Anotácia `@Path("skola")` triedy zdroja definuje relatívnu URI koreňového zdroja v hierarchii. Požiadavka GET na koreň vráti názov školy.

Adresa školy predstavuje samostatný pod-zdroj. Relatívna URI pod-zdroja je definovaná anotáciou prístupovej metódy `@Path("adresa")` .

Teda URI adresy školy je `<ip-servera>/<aplikacia>/resources/skola/adresa`.

Fragmenty URI zdroja sú parametrizovateľné. Jedna metóda môže tak realizovať prístup k viacerým zdrojom. Informácie o predmetoch môže poskytovať nasledujúca metóda.

```
    @Path("predmet/{nazov}")
    @Produces(MediaType.TEXT_PLAIN)
    public String getPredmet(@PathParam("nazov") String name) {
       if ("VSA".equals(name))return "Informácie k predmetu VSA ...";
       if ("OOP".equals(name))return "Informácie k predmetu OOP ...";
// a.t.d.
       return null;
    }
```

Parametrizovateľný fragment URI v anotácii `@Path("predmet/{nazov}")` je uzavretý v zložených zátvorkách. Jeho hodnota, názov predmetu, je anotáciou `@PathParam` injektovaná ako argument metódy. Informácie o predmete VSA budú teda prístupné pod URI:

`<ip-servera>/<aplikacia>/resources/skola/predmet/VSA`.

Trieda Response

Keď metóda zdroja vráti hodnotu null, klientovi sa odošle odpoveď so status kódom `404 - Not Found`. Ak by sme chceli poslať odpoveď s iným kódom, napr. `204 - No Content`, je potrebné, aby metóda vracala inštanciu triedy `Response`. Táto trieda umožňuje zadať nielen status kód a obsah odpovede, ale tiež vložiť metadáta do hlavičky.

```
    @GET
    @Produces(MediaType.TEXT\_PLAIN)
    @Path("predmet/{nazov}}")
    public Response getPredmet(@PathParam("nazov") String name) {

        if ("VSA".equals(name)) {
            return Response
                    .status(Response.Status.OK)
                    .entity("Informacie o predmete VSA")
                    .build();
        }
// a.t.d.
        return Response
                .status(Response.Status.NO_CONTENT)
                .build();
    }
```

Výber metódy na spracovanie HTTP požiadavky

Pre konkrétnu URI a HTTP požiadavku, môže teda trieda poskytovať aj viacero metód, musia sa však odlišovať aspoň MIME-typom v anotáciách `@Produces`, resp. `@Consumes`.

V našom príklade môžeme pre koreňový zdroj okrem GET-metódy, ktorá vracia názov školy (ako jednoduchý text) implementovať aj druhú, ktorá vráti obrázok s logom školy.

```
    @GET
    @Produces("image/*")
    public Response getLogo() {
        File f = new File("/home/edu/Pictures/feistu.jpg");
        if (!f.exists()) {
            throw new WebApplicationException(404);
        }
        String mt = new MimetypesFileTypeMap().getContentType(f);
        return Response.ok(f, mt).build();
    }
```

Pri výbere metódy pre spracovanie konkrétnej HTTP požiadavky sa postupuje podľa nasledujúcich zásad:

- URI požiadavky sa porovná s URI vytvorenými na základe `@Path` anotácie triedy a metódy.
- HTTP metóda požiadavky sa porovná s anotáciou metódy zdroja (`@GET @POST`,...)
- Formát reprezentácie, ktorý požaduje klient (nachádza sa v hlavičke `Accept` požiadavky) porovná s MIME-typom v anotácii `@Produces`.
- Formát obsahu požiadavky (nachádza sa v hlavičke `Content-type`) sa porovná s MIME-typom v anotácii `@Consumes`.

Pozri Oracle JAX-RS tutorial[16].

Ktorá z metód v našom príklade spracuje požiadavku GET na koreňový zdroj (čiodpoveďou bude text alebo obrázok), záleží teda od toho, čo klient očakáva. Pozri cvičenie 7.4.

Poznámka: Metóda GET, ktorá vracia súbor (napr. obrázok) ako reprezentáciu zdroja, na to použije opäť triedu `Response`. Tá umožňuje zoserializovať súbor do obsahu správy a do hlavičky vložiť jeho formát, príp. ďalšie metadáta.

Implementáciu príkladu 18 nájdete v projekte **rs1skola** na disku STU.

[16] https://www.oracle.com/technical-resources/articles/java/jax-rs.html

Cvičenia

Cvičenie 7.1

V projekte **rs1simple** odstráňte z triedy zdroja anotáciu `@ApplicationScoped` a presvedčte sa pomocou klientskej aplikácie, že teraz nie je možné zmeniť text správy pomocou PUT.

Cvičenie 7.2 Využitie kontextu servleta

Ak nechceme/nemôžeme zabezpečiť (pomocou vhodnej anotácie), že sa pre každú HTTP požiadavku nebude vytvárať nová inštancia triedy zdroja, môžeme stav zdroja (aktuálnu správu) uložiť v kontexte servleta REST služby. Tento kontext sa vytvára len raz pri spustení web aplikácie. Referenciu na neho môžeme injektovať ako argument metódy zdroja pomocou anotácie `@Context ServletContext`.

Upravte a otestujte implementáciu triedy zdroja v projekte **rs1simple**:

```
@Path("message")
public class SimpleResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getText( @Context ServletContext ctx) {
        return ctx.getAttribute("sprava");
    }

    @PUT
    @Consumes(MediaType.TEXT_PLAIN)
    public void putText(@Context ServletContext ctx, String content ) {
        ctx.setAttribute("sprava", content);
    }
}
```

Cvičenie 7.3 Prístup k súborom na serveri

GET metóda `getLogo` v príklade 18 (projekt **rs1skola**), číta obrázok zo súboru na lokálnom súborovom systéme. Pri nasadzovaní služby na aplikačný server treba s ňou preniesť aj súbor s obrázkom. Preto je potrebné umiestniť obrázok do adresára projektu. V čase implementácie síce nepoznáme absolútnu cestu koreňového adresára webovej aplikácie na aplikačnom serveri, ale kontext REST servleta ju umožňuje získať.

```
    @GET
    @Produces("image/*")
    public Response getLogo( @Context ServletContext ctx ) {
        File f = new File(ctx.getRealPath("feistu.jpg"));
```

Umiestnite súbor s obrázkom priamo do koreňového adresára webovej aplikácie (je tam aj index.html), upravte implementáciu metódy `getLogo` a otestujte ju.

Cvičenie 7.4 Výber metódy pre spracovanie požiadavky

V príklade 18 (pozri projekt **rs1skola**) trieda zdroja poskytuje dve GET metódy pre koreňové URI `skola`, jedna vracia textovú reprezentáciu a druhá obrázok.

a) Zistite, ktorá metóda spracuje GET požiadavku na koreňové URI odoslanú z prehliadača. T.j. či dostanete textovú reprezentáciu alebo obrázok. Skúste rôzne prehliadače, príp. iné klientské aplikácie.

b) V anotácii metódy zdroja `getInfo()` zmeňte formát z `TEXT_PLAIN` na `TEXT_HTML` a klientom opäť overte, ktorú reprezentáciu dostanete teraz.

Vysvetlenie: Pri výbere metódy pre spracovanie požiadavky sa zohľadňuje hodnota poľa `accept` v hlavičke požiadavky. Pre prístup k hlavičke pozri cvičenie 7.8

Cvičenie 7.5 Relatívna URI zdroja definovaná regulárnym výrazom

Parametrizovaný fragment URI v anotácii `@Path` môže obsahovať regulárny výraz, ktorý definuje platné hodnoty fragmentu. Do projektu **rs1skola** pridajte GET metódu, ktorá vracia informácie o triedach gymnázia.

```
    @Path(trieda/{nazov: [1-4][A-C]})
    @Produces(MediaType.TEXT_PLAIN)
    public String getTrieda(@PathParam("nazov") String trieda) {
       return "Informácie o triede " + trieda;
    }
```

Presvedčte sa, že platné sú len názvy tried odpovedajúce regulárnemu výrazu `[1-4][A-C]`.
Zistite, akú odpoveď vráti GET požiadavka na triedu s neplatným názvom.

Cvičenie 7.6 Číslo ako relatívna URI zdroja

Ak je parametrizovaný fragment URI v anotácii `@Path`, číslo, môžeme dať argumentu metódy číselný typ. Do projektu **rs1skola** pridajte GET metódu, ktorá vracia informácie o ročníkoch, pričom platné ročníky sú čísla 1-9.

```
    @Path({rocník})
    @Produces(MediaType.TEXT_PLAIN)
    public String getTrieda(@PathParam("rocnik") int n) {
       if (n>0 && n<10) {
          return "Informacie o rocniku " + n;
       }
       return "Neplatny rocnik " + n;
    }
```

Presvedčte sa, že platné URI sú len tie, v ktorých je fragment `rocník` číslo `1-9`. Všimnite si tiež, že konverzia na číslo sa urobí automaticky pri extrakcii fragmentu z URI a zistite, akú HTTP odpoveď vráti požiadavka, ak ročník v URI nie je číslo.

Cvičenie 7.7 Parametre dotazu

Metódy zdroja môžu získať a použiť aj parametre HTTP požiadavky. Hodnota parametra je injektovaná ako argument metódy anotáciou `@QueryParam("meno")`, kde `meno` je identifikátor parametra, ako ilustruje jednoduchý príklad služby:

```
@Path("pozdrav")
public class PozdravResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getText(@QueryParam("meno") String name) {
        return "Dobré ráno " + name;
    }
}
```

Príklad implementujte a overte si odpoveď na GET požiadavku pre URI
`<ip-servera>/<aplikacia>/resources/pozdrav?meno=Fero`.

Doplňte deklaráciu metódy o definíciu náhradnej hodnoty pre parameter požiadavky.

```
    public String getText(@DefaultValue("neznámy") @QueryParam("meno") String
        name) {
        return "Dobré ráno " + name;
```

Presvedčte sa, že odpoveď na GET požiadavku bez parametra `meno` obsahuje náhradnú hodnotu.

Cvičenie 7.8 Hlavička dotazu

Pomocou anotácie `@Context` je možné injektovať ako argument metódy zdroja tiež objekt triedy `HttpHeaders`. Prostredníctvom neho má metóda prístup ku všetkým údajom v hlavičke HTTP požiadavky.
Implementujte nasledujúcu GET metódu a presvedčte sa, že vráti obsah hlavičky dotazu.

```
    @GET
    @Produces("text/html")
    public String getHeader(@Context HttpHeaders hh) {
        MultivaluedMap<String, String> headerParams = hh.getRequestHeaders();
        String resp = "";
        for(String key : headerParams.keySet()) {
            resp = resp + key + " = " + headerParams.getFirst(key) + "<br/>";
        }
        return resp;
    }
```

8 REST a JPA

8.1 Vytváranie a odstraňovanie zdrojov

Ako sme v úvode spomínali, REST-API poskytuje tiež metódy na vytváranie nových resp. odstraňovanie existujúcich zdrojov.

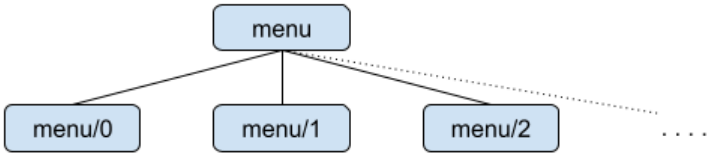
Pri návrhu služby umožňujúcej vytvárať zdroje je potrebné rozhodnúť, či URI nového zdroja definuje server alebo klient:

- Ak URI definuje server, klient posiela reprezentáciu nového zdroja požiadavkou `POST` na URI kolekcie, do ktorej nový zdroj pridáva.
- Ak URI nového zdroja definuje sám klient, potom posiela jeho reprezentáciu priamo na URI nového zdroja požiadavkou `PUT`.

Rôzne varianty a ich implementácie si ilustrujeme na príklade webovej služby jedálne, ktorá bude poskytovať aktuálne informácie o ponuke jedál. Zároveň umožňuje vkladať a odstraňovať jedlá z ponuky.

Príklad 19: REST služba jedálne - varianta POST

Ponuka jedálne predstavuje kolekciu jedál a je prístupná ako koreňový zdroj s URI `menu`. Jednotlivé jedlá v ponuke dňa budú prístupné ako pod-zdroje s URI `menu/{n}`, kde `n` je poradové číslo jedla v ponuke, pričom jedlá sú číslované za sebou od 0.



Pre URI ponuky dňa `menu` poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti aktuálny počet jedál v menu.
- **POST** pre MIME `text/plain`: pridá nové jedlo do kolekcie menu.

Pre URI jedla `menu/{n}` poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti názov jedla s poradovým číslom `n`.
- **DELETE**: odstráni jedlo s poradovým číslom `n`.
  Poznámka: Po odstránení jedla sa poradové čísla jedál môžu zmeniť.

Implementácia:

Názvy jedál v ponuke budeme v pamäti udržiavať ako zoznam reťazcov: `List<String>`. Index v zozname môžeme použiť priamo ako jeho relatívnu URI jedla. Implementácia metód služby je jednoduchá, možno volať priamo metódy zoznamu.

```
@Singleton
@Path("menu")
public class JedalenResource {
    private List<String> jedla;
    public JedalenResource() {
        jedla = new ArrayList<>();
    }

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getPocet() {
        return "" + jedla.size();
    }

    @POST
    @Consumes(MediaType.TEXT_PLAIN)
    public void pridajJedlo(String content) {
        jedla.add(content);
    }

    @GET
    @Path("{cislo}")
    @Produces(MediaType.TEXT_PLAIN)
    public String getJedlo(@PathParam("cislo") int n) {
```

```
        if (n < 0 || n >= jedla.size()) {
            return null;
        }
        return jedla.get(n);
    }

    @DELETE
    @Path("{cislo}")
    public void odstranJedlo(@PathParam("cislo") int n) {
        if (n < 0 || n >= jedla.size()) {
            return;
        }
        jedla.remove(n);
    }
}
```

Poznámky:

- Ak by mali byť jedlá v ponuke číslované od 1, museli by sme k indexu pripočítať 1, pretože zoznam je indexovaný od 0.
- Pri odstránení položky zo zoznamu sa položky za ňou posunú a tak sa ich relatívne URI znížia, s tým však špecifikácia počíta.
- Táto jednoduchá REST služba nemá metódu PUT na modifikáciu jedál. Modifikáciu jedla môžeme urobiť nepriamo, tak, že ho odstránime a znovu vytvoríme. Pritom sa však zmení jeho URI. (Implementácia bez zmeny URI tiež síce možná, pri použití zoznamu však trochu zložitejšia.)
- Pripomeňme tiež, že POST sa aplikuje vždy na URI kontajnera. Od klienta dostane len reprezentáciu nového zdroja, nie však jeho URI. URI novovytvoreného zdroja (v našom prípade poradové číslo nového jedla) sa definuje na serveri, klient sa ho však nedozvie. Preto by bolo vhodné, keby POST relatívne URI vrátil.

Implementácia príkladu 19 s metódou POST, ktorá vracia číslo jedla nového, je v projekte **JedalenPost** na disku STU.

Príklad 20: REST služba jedálne - varianta PUT

Ponuku jedálne predstavuje opäť koreňový zdroj s URI `menu`. Zdroj reprezentujúci konkrétne jedlo v ponuke bude mať URI `menu/{nazov}`, kde `nazov` je názov jedla. Jeho textová reprezentácia môže teraz poskytovať popis jedla s ďalšími informáciami o ňom (cena, alergény...).

Pre URI ponuky dňa `menu` poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti reťazec obsahujúci názvy jedál oddelené medzerou.
  Poznámka. Metóda pre `POST` nie je implementovaná. Namiesto toho sa bude nový zdroj pridávať požiadavkou `PUT` pre URI nového zdroja - jedla.

Pre URI jedla `menu/nazov` poskytuje služba metódy:

- **GET** pre MIME `text/plain`: vráti popis jedla s daným názvom.
- **PUT** pre MIME `text/plain`: zmení popis jedla s daným názvom alebo pridá nové jedlo (náhrada za POST)
- **DELETE**: odstráni jedlo s daným názvom.

Poznámka k implementácii:

Pre uloženie informácií o jedlách v pamäti môžeme použiť kontajner `Map<String,String>`, kde kľúč bude názov jedla a hodnota jeho popis.

Implementáciu príkladu 20 nájdete v projekte **JedalenPut** na disku STU.

8.2 Zdroje so štrukturovanými dátami

REST služby umožňujú posielať medzi klientom a serverom aj štruktúrované dáta, a to vo formáte XML alebo JSON.

Prácu so štruktúrovanými údajmi si ilustrujeme opäť na REST službe jedálne. Obe varianty implementácie služby sa dajú upraviť tak, že reprezentácia jedla bude štruktúrovaná.

Príklad 21: REST služba jedálne - varianta POST XML

Reprezentácia jedla vo formáte XML bude mať štruktúru:

```
    <jedlo>
        <nazov>gulas</nazov>
        <cena>3.5</cena>
    </jedlo>
```

Poznámka: Štruktúru XML dokumentov budeme vo výklade zvyčajne popisovať len neformálne, pomocou ilustračných ukážok/príkladov. XML dokumenty, s ktorými pracuje služba, sa môžu líšiť textovým obsahom, príp. aj počtom elementov.

Rovnako ako vo variante z príkladu 19 budú jednotlivé jedlá v ponuke prístupné ako zdroje s URL `menu/n`, kde `n` je poradové číslo jedla v ponuke, pričom jedlá sú číslované za sebou od 0.

Pre URI ponuky `menu` poskytuje služba metódy

- **GET**: Vráti aktuálny počet jedál v menu - pre MIME `text/plain`.
- **POST**: Pridá nové jedlo do menu.
  Akceptuje XML s informáciami o novom jedle - MIME `application/xml`.
  Vráti poradové číslo jedla - MIME `text/plain`.

Pre URI jedla `menu/{n}` poskytuje metódy

- **GET**: Vráti XML s informáciami o jedle - MIME `application/xml`.
- **DELETE**: Odstráni jedlo s poradovým číslom `n`.

Implementácia

Pre reprezentáciu informácií o jedle definujeme dátovú triedu `Jedlo`. Technológia JAX-RS umožňuje (prostredníctvom komponentu JAXB) automatickú serializáciu (zápis) a deserializáciu (načítanie) dátových objektov do/z XML alebo JSON formátu. Stačí, ak má dátová trieda anotáciu `@XmlRootElement` a štandardnú java-bean štruktúru.

```
@XmlRootElement
public class Jedlo implements Serializable {
    private String nazov;
    private double cena;
// gettre a settre...
```

Poznámka: Názov koreňového elementu XML dokumentu bude odvodený od názvu triedy, názvy podelementov sú odvodené od názvov dátových členov. Toto štandardné mapovanie je možné zmeniť. JAXB poskytuje celý rad anotácií, ktorými možno prispôsobiť názvy elementov a štruktúru dokumentu vytvoreného serializáciou dátovej triedy. Pozri napr. JAXB user guide[17].

Implementácia triedy zdroja `JedalenResource` je podobná ako v projekte **JedalenPost**. Stačí nahradiť zoznam reťazcov zoznamom `List<Jedlo>` a upraviť implementáciu metód POST a GET:

- MIME-type treba zmeniť na `application/xml` alebo `application/json`.
- Argument metódy POST a návratová hodnota metódy GET budú typu `Jedlo`.

  Implementáciu príkladu 21 nájdete v projekte **JedalenPostPojo** na disku STU.

Podobným spôsobom môžeme modifikovať aj variantu služby z príkladu 20, ktorá používa HTTP požiadavku PUT na pridávanie, resp. modifikáciu jedál. Pozri cvičenie 8.7.

Implementácia klienta v jazyku Java

Štruktúrované dáta, ktoré posiela REST služba vo formáte XML alebo JSON, môže klient vždy načítať ako textový dokument. Napríklad klient služby z príkladu 21 by mohol pre získanie informácií o jedle použiť jednoduchú funkciu:

```
public void getJedlo(int n) {
    String BASE_URI = "http://localhost:8080/JedalenPostPojo/resources";
    Client client = ClientBuilder.newClient();
    WebTarget rootResource = client.target(BASE_URI).path("menu");
    WebTarget subResource = rootResource.path("" + n);
    String result = subResource.request(MediaType.APPLICATION_XML).get(String.
        class);

    System.out.println("GET returned: " + result);
}
```

[17] https://javaee.github.io/jaxb-v2/doc/user-guide/ch03.html#annotating-your-classes

Funkcia očakáva odpoveď na GET vo formáte XML: `.request(MediaType.APPLICATION_XML)`, ale odpoveď načíta ako reťazec: `.get(String.class)`.

Samozrejme, ak máme k dispozícii dátovú triedu použitú na strane servera, môžeme ju použiť aj na strane klienta. Volanie požiadavky GET môžeme potom upraviť tak, aby sa odpoveď automaticky načítala do inštancie dátovej triedy.

```
Jedlo result = subResource.request(MediaType.APPLICATION_XML).get(Jedlo.
    class);
```

Funkciu na pridanie nového jedla môžeme na strane klienta implementovať nasledovne:

```
public String pridajJedlo(Jedlo jedlo) {
    String BASE_URI = "http://localhost:8080/JedalenPostPojo/resources";
    Client client = ClientBuilder.newClient();
    WebTarget rootResource = client.target(BASE_URI).path("menu");
    return webTarget.request(MediaType.TEXT_PLAIN)
        .post(Entity.entity(jedlo,MediaType.APPLICATION_XML), String.class)
}
```

Informácie o jedle posiela klient požiadavkou POST na zdroj `menu` vo formáte XML: `.entity(jedlo,MediaType.APPLICATION_XML)`. Odpoveď očakáva v textovom formáte: `.request(MediaType.TEXT_PLAIN)`

Poznámka:

Pre implementáciu klienta poskytuje prostredie NetBeans nástroj *RESTful Java Client*, ktorý vygeneruje proxy triedu s funkciami uľahčujúcimi volania REST služby. Podobné nástroje majú aj iné vývojové prostredia.

8.3 Integrácia REST a JPA

Dátové objekty reprezentujúce stav zdroja môžu byť na strane servera persistentne uložené v relačnej databáze. Na prístup k databáze a synchronizáciu ich stavu je vhodné použiť JPA.

Konfigurácia JPA pre web aplikáciu

Databázové web-aplikácie bežiace na aplikačnom servery môžu využívať transakčný mechanizmus **JTA** poskytovaný aplikačným serverom, ktorý automaticky otvára a ukončuje samostatnú databázovú transakciu pre každú HTTP požiadavku.

Pre použitie transakčného mechanizmu **JTA** potrebné upraviť súbor `persistence.xml`:

```
<persistence-unit name="my_persistence_unit" transaction-type="JTA">
    <jta-data-source>jdbc/sample</jta-data-source>
 </persistence-unit>
```

Element `jta-data-source` obsahuje identifikátor konfigurácie s údajmi pre pripojenie k databáze. Táto konfigurácia sa však nenachádza priamo v súbore `persistence.xml`, ale je definovaná v rámci konfigurácie aplikačného servera.

Triedu zdroja vytvoríme ako EJB komponent, čo dosiahneme napríklad anotáciou `@Stateless`. Referenciu na manažér entít môžeme potom do triedy zdroja injektovať anotáciou `@PersistenceContext`.

```
import jakarta.ejb.Stateless;
@Stateless
@Path("menu")
public class JedalenResource {
    @PersistenceContext(unitName = "my_persistence_unit")
    private EntityManager em;
// ...
```

S využitím manažéra entít môžu metódy služby synchronizovať stav zdrojov okamžite s databázou.

Príklad 22: REST služba jedálne - varianta POST s ukladaním v databáze

Implementácia:

Informácie o jedlách budú persistentne uložené v databázovej tabuľke `JEDLO`. Dátová trieda `Jedlo` je teda entitnou triedou. Namiesto indexu v zozname bude teraz relatívnou URI jedla hodnota kľúča typu `long`. Keďže na vytváranie nových zdrojov použijeme v tejto variante metódu POST, kľúč predstavujúci relatívnu URI nového zdroja je definovaný serverom. Môže byť teda generovaný priamo databázou.

```
@Entity
@XmlRootElement
public class Jedlo implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    private String nazov;
    private double cena;
// ...
```

Implementácia metód zdroja je priamočiara. Napríklad GET metódu, ktorá vracia informácie o konkrétnom jedle, môžeme implementovať jednoduchým volaním metódy `find` manažéra entít.

```
    @GET
    @Path("{cislo}")
    @Produces(MediaType.APPLICATION_XML)
    public Jedlo getJedlo(@PathParam("cislo") long id) {
        return em.find(Jedlo.class, id);
    }
```

Implementáciu príkladu 22 nájdete v projekte **JedalenPostDB** na disku STU.

Cvičenia

Cvičenie 8.1 Implementujte java klientov pre varianty REST služby jedálne z príkladov 19, 20 a 21, ktorými otestujete pridávanie a odstraňovanie jedál.

Cvičenie 8.2 REST služba jedálne - použitie formátu JSON

Upravte príklad 20 (projekt **JedalenPut**) tak, aby požiadavka GET pre koreňový zdroj `menu` vrátila namiesto jedného reťazca zoznam reťazcov s názvami jedál vo formáte JSON.

Návod: Návratovú hodnotu metódy zdroja zmeňte na `java.util.list` a typ média v anotácii na `APPLICATION_JSON`.

Všimnite si, že zoznam bude automaticky serializovaný do formátu JSON.
Poznámka. Vo formáte XML zoznam reťazcov nie je možné vrátiť.

Cvičenie 8.3 REST služba jedálne - GET vracajúca zoznam objektov

Do projektu **JedalenPostPojo** pre koreňový zdroj doplňte metódu pre požiadavku GET, ktorá vráti zoznam všetkých jedál vo formáte

a) JSON
b) XML

Poznámka: Služba bude teda pre kolekciu `menu` poskytovať dve GET metódy. To je možné, keďže vracajú reprezentáciu zdroja v rôznych formátoch. Pomocou prehliadača otestujte obe metódy. V prípade b) si všimnite názov koreňového XML-elementu v odpovedi ☺.

Cvičenie 8.4 REST služba jedálne - doplnenie metódy PUT do varianty POST XML

Do projektu **JedalenPostPojo** doplňte implementáciu metódy pre požiadavku PUT, ktorá umožní klientovi modifikovať cenu, prípadne názov jedla v ponuke.

Pozor: Na rozdiel od POST sa požiadavka PUT posiela na URI konkrétneho jedla a zmení jeho stav (cenu,...). Nevytvára nový zdroj. Implementujte ju preto tak, aby nezmenila relatívnu URI zdroja.

Cvičenie 8.5 REST služba jedálne - doplnenie metódy PUT do varianty POST s ukladaním v databáze

Do projektu **JedalenPostDB** doplňte implementáciu metódy pre požiadavku PUT, ktorá umožní klientovi modifikovať cenu, prípadne názov jedla v ponuke.

Cvičenie 8.6 REST služba jedálne - modifikácia varianty POST s ukladaním v databáze

Do projektu **JedalenPostDB** doplňte implementáciu GET metódy pre `menu`, ktorá vráti zoznam číselných kľúčov (teda relatívnych URI) jedál aktuálne v menu vo formáte **JSON**.

Cvičenie 8.7 REST služba jedálne - varianta PUT XML

Reimplementujte príklad 20 (projekt **JedalenPut**) tak, aby informácie o jedle boli v pamäti reprezentované ako objekty triedy `Jedlo` a posielané vo formáte XML.

```
public class Jedlo {
    private String popis;
    private double cena;
    private List<Short> alergen;
```

Všimnite si, akým spôsobom bude zoznam alergénov serializovaný do XML.

Cvičenie 8.8 REST služba jedálne - varianta PUT s ukladaním v databáze

Reimplementujte projekt z cvičenia 8.7 tak, aby informácie o jedlách boli ukladané v databáze.
Pozor: Do triedy `Jedlo` je treba doplniť kľúč - názov jedla:

```
public class Jedlo {
    @Id
    private String nazov;
    private String popis;
    private double cena;
    private List<Short> alergen;
```

Cvičenie 8.9 Overenie funkčnosti REST služby z cvičenia 8.8

Pri posielaní požiadavky PUT na službu jedálne s ukladaním v databáze (cvičenie 8.8), zadáva klient názov jedla ako relatívnu URI, ale môže ho zadať v XML-elemente `nazov`, keďže `nazov` je kľúčový dátový člen entitnej triedy `Jedlo`.

Implementujte klientskú aplikáciu, ktorá preverí spracovanie PUT požiadavky, keď klient

a) v XML-elemente `nazov` zadá iný názov pre jedlo ako v URI,
b) XML-element `nazov` úplne vynechá.

Aplikácia najprv pošle PUT požiadavku pre vytvorenie nového jedla a následne si GET požiadavkou preverí, či bolo nové jedlo vytvorené, má požadované URI a stav.

Odporúčanie:
Aby sa vyhlo problémom spôsobeným klientom, ktorý zadá iný názov jedla do URI a do obsahu PUT správy, mala by metóda služby použiť ako kľúč v databáze hodnotu uvedenú v URI. Hodnotu, ktorú klient zadá do XML-elementu, môže metóda ignorovať, resp. klient ju vôbec nemusí posielať.
To môžeme dosiahnuť anotáciou `@XmlTransient` dátového člena `nazov`.

Cvičenie 8.10 REST služba jedálne s menu pre celý týždeň
Menu s ponukou pre jeden deň bude prístupné ako zdroj s URI `menu/{den}` kde `den` je meno dňa v týždni. Platné mená dní sú reťazce: `pondelok, utorok, streda, stvrtok, piatok`.
Zdroj reprezentujúci konkrétne jedlo v ponuke dňa bude mať URI `menu/{den}/{nazov}`, kde `nazov` je názov jedla. Jeho reprezentácia bude mať formát XML so štruktúrou ako v cvičení 8.7.

Pre URI ponuky dňa `menu/{den}` poskytuje služba metódu:
- **GET** - vráti zoznam názvov jedál v ponuke dňa vo formáte JSON.

Pre URI jedla `menu/{den}/{nazov}` poskytuje služba metódy:
- **GET** pre MIME `APPLICATION_XML` - vráti reprezentáciu jedla vo formáte XML.
- **PUT** pre MIME `APPLICATION_XML` - aktualizuje informácie o jedle.
- **DELETE** - odstráni jedlo s daným názvom.

Metóda **GET** pre koreňové URI `menu` umožňuje tiež zistiť, ktorý deň je jedlo v ponuke. Názov jedla zadá klient ako parameter požiadavky (`@QueryParam("nazov")`). Zoznam dní, keď je jedlo s daným názvom v ponuke, vráti metóda vo formáte JSON.

a) Implementujte službu tak, že všetky informácie budú udržiavané len v pamäti, (použite vhodné dátové štruktúry).
Službu inicializujte tak, že pri jeho spustení bude už menu pre každý deň obsahovať pizzu za 6.50 (ako stálu ponuku).
b) Pomocou vhodného klienta službu otestujte: Metódou PUT zmeňte cenu pizze v pondelkovom menu a pozrite sa, či sa pri tom zmenila cena pizze aj v ostatných dňoch.

Cvičenie 8.11 REST služba jedálne s menu pre celý týždeň s ukladaním v databáze
Reimplementujte REST službu z cvičenia 8.10 tak, že informácie o menu pre celý týždeň budú uložené v databáze.
Implementáciu služby otestujte klientom rovnako ako v cvičení 8.10. Porovnajte, či sa obe implementácie služby správajú rovnako pri zmene ceny pizze.

Cvičenie 8.12 REST služba jedálne s menu pre celý týždeň s ukladaním v databáze - varianta POST
Implementujte službu jedálne s menu pre celý týždeň s ukladaním v databáze, pričom jedlá budú do menu vkladané pomocou metódy POST. (Teda ich identifikátor bude kľúč generovaný databázou, podobne ako v projekte **JedalenPostDB**.) Pre URI `menu/{den}/{id}` implementujte tiež metódu PUT pre modifikáciu informácií o jedle.
Implementáciu služby otestujte klientom rovnako ako v cvičeniach 8.10 a 8.11 . Porovnajte ich správanie pri zmene ceny pizze.

9 Otázky a problémy pri použití REST

V tejto kapitole ilustrujeme na príkladoch viaceré otázky, s ktorými sa môžeme stretnúť pri návrhu, implementácii a používaní REST služieb.

- Ako môžu klienti zistiť URI zdrojov?
- Ako môže zistiť klient vnútornú štruktúru XML/JSON dát, ktoré akceptuje služba?
- Môže byť jeden zdroj prístupný prostredníctvom rôznych URI?

9.1 Služba z pohľadu klienta

Pozrime sa na REST službu jedálne z pohľadu klienta, resp. vývojára klientskej aplikácie.

Ako môže klient zistiť URI zdrojov?

Aby klient mohol používať REST službu, musí poznať aspoň URI zdrojov, ktoré sprístupňuje. Informácie o zdrojoch a prístupových metódach sú po spustení služby vypublikované v tzv. WADL dokumente pod URI: `<ip-servera>/<aplikacia>/resources/application.wadl`. Pozri špecifikáciu jazyka WADL[18].

Vo WADL dokumente REST služby jedálne z **príkladu 19** nájdeme fragment s informáciami o koreňovom zdroji:

```
<resources base="http://localhost:8080/JedalenPost/resources/">
  <resource path="menu">
    <method id="getPocet" name="GET">
      <response>
        <representation mediaType="text/plain"/>
      </response>
    </method>
    <method id="pridajJedlo" name="POST">
      <request>
        <representation mediaType="text/plain"/>
      </request>
      <response>
        <representation mediaType="text/plain"/>
      </response>
    </method>
```

Z fragmentu môžeme vyčítať, že relatívna URI koreňového zdroja je `menu` a pre narábanie s ním poskytuje REST služba HTTP metódy GET a POST, ktoré posielajú resp. akceptujú textové dáta.

Z ďalšej časti WADL dokumentu môžeme zistiť, že koreňový zdroj `menu` je kolekcia a zdroje v tejto kolekcii majú relatívnu URI, ktorá je číslo (viď XML-element `param`):

```
    <resource path="{cislo}">
      <param name="cislo" style="template" type="xs:int"/>
      <method id="odstranJedlo" name="DELETE"/>
      <method id="getJedlo" name="GET">
        <response>
          <representation mediaType="text/plain"/>
        </response>
      </method>
    </resource>
```

Ako klient zistí URI zdrojov v kolekcii?

Klient môže poslať požiadavku GET pre koreňový zdroj `menu` a v odpovedi dostane informáciu o aktuálnom počte jedál (pozri špecifikáciu príkladu 19). Z toho by mohol usúdiť, že relatívne URI jedál by mohli byť ich poradové čísla. Stále však nie je jasné, či sa číslovanie začína od 0 alebo od 1. Overiť sa to dá len ďalším experimentovaním.

Ešte zložitejšie to je však v príklade **22**, kde sú informácie o jedlách uložené v databáze, pričom relatívna URI jedla je číselný kľúč generovaný databázou. Číselné kľúče tu nemusia ísť za sebou a tak klient len ťažko uhádne URI jedál v ponuke.

Riešením by bolo, keby GET metóda pre `menu` vrátila zoznam kľúčov jedál v menu (teda relatívnych URI zdrojov v kolekcii). Pozri cvičenie 8.6.

[18] https://www.w3.org/submissions/wadl/

Ako klient zistí vnútornú štruktúru XML/JSON dát?

WADL súbor poskytuje klientom popri URI zdrojov aj formát (MIME-type) reprezentácie ich stavu pre jednotlivé pristupové metódy. V prípade štruktúrovaných dát vo formátoch JSON alebo XML však neposkytuje žiadnu ďalšiu informáciu o ich vnútornej štruktúre. Ak nemáme k dispozícii špecifikáciu dátových tried na strane servera, nevieme celkom dobre vytvoriť dátovú triedu, do ktorej by sme mohli dáta posielané vo formáte XML alebo JSON načítať. Môžeme ich načítať ako textový reťazec. Pomocou volania GET tak získať nejaké príklady dát a podľa nich skúsiť navrhnúť vhodnú dátovú triedu.

Ilustrujeme si tento postup pri návrhu a implementácii klienta REST služby jedálne - varianta PUT s ukladaním v databáze. (Implementácia služby bola úlohou na cvičenia 8.8.)

Predpokladajme, že služba je spustená a v databáze sa už nachádzajú dve jedlá:

- Guláš s cenou 6.5 bez alergénov a popisu
- Špagety s cenou 5.5 a alergénom 7, bez popisu

GET metóda pre kolekciu `menu` nám vráti JSON obsahujúci názvy jedál v ponuke:

```
["Gulas","Spagety"]
```

Volanie GET pre podzdroj `menu/Gulas` vráti informácie o tomto jedle vo formáte XML:

```
<jedlo>
    <nazov>Gulas</nazov>
    <cena>6.5</cena>
</jedlo>
```

podobne pre `menu/Spagety`

```
<jedlo>
    <nazov>Spagety</nazov>
    <cena>5.5</cena>
    <alergen>7</alergen>
</jedlo>
```

Na základe týchto experimentov by sme pre implementáciu klienta mohli navrhnúť triedu `Jedlo` s troma dátovými členmi.

```
class Jedlo implements Serializable {
    private String nazov;
    private double cena;
    private int alergen;
```

V skutočnosti však informácia o jedle môže mať aj ďalšiu položku - popis. Navyše, alergénov tam môže byť aj viac. Správne navrhnutá trieda by mala vyzerať tak, ako na strane servera.

```
class Jedlo implements Serializable {
    private String nazov;
    private String popis;
    private double cena;
    private List<Short> alergen;
```

Detailné informácie o štruktúre XML reprezentácie zdroja však WADL dokument neobsahuje. Pokiaľ nemáme k dispozíci inú podrobnú dokumentáciu k službe, môžeme navrhnúť triedu len na základe experimentov, pozri cvičenie 8.7.

9.2 REST služba s komplexnou štruktúrou zdrojov

Ďalšie problémy a otázky si ilustrujeme na REST službe s viacúrovňovou hierarchiou zdrojov, ktorá umožňuje klientom pracovať a navigovať medzi zdrojmi podľa rôznych scenárov.

Príklad 23: Portál Svet kníh

Portál Svet kníh je REST služba, ktorá poskytuje informácie o kníhkupectvách a knihách v ponuke.

Špecifikácia služby pre portál

Informácie o kníhkupectve:

- názov - je jeho jednoznačná identifikácia
- ponuka - obsahuje zoznam ponúkaných kníh

- adresa

Informácie o knihe:

- isbn - jednoznačný identifikátor knihy
- titul
- cena

Koreňový zdroj portálu reprezentuje kolekciu kníhkupectiev a má URI `SvetKnih`.
Kníhkupectvá predstavujú samostatné zdroje s URI `SvetKnih/{obchod}`, kde `obchod` je názov kníhkupectva.
Informácie o knihách v ponuke kníhkupectva predstavujú zdroje s URI `SvetKnih/{obchod}/{isbn}`, kde `isbn` je jednoznačný identifikátor knihy.

Pre prístup ku zdrojom poskytuje služba nasledujúce metódy:
Pre `SvetKnih`

- GET vráti zoznam názvov obchodov vo formáte JSON.

Pre `SvetKnih/{obchod}`

- GET vráti všetky informácie o obchode a ponúkaných knihách vo formáte XML.

Pre `SvetKnih/{obchod}/{isbn}`

- GET vráti všetky informácie o knihe vo formáte XML.
- PUT - umožní modifikovať informácie o knihe. Akceptuje informácie o knihe vo formáte XML.

Implementácia dátových tried

Dátové triedy by mohli vyzerať nasledovne:

```
public class Kniha {
    private String isbn;
    private String titul;
    private double cena;
```

```
public class Knihkupectvo {
    private String nazov;
    private String adresa;
    @XmlElementWrapper(name="ponuka")
    @XmlElement(name="kniha")
    private List<Kniha> ponuka;
```

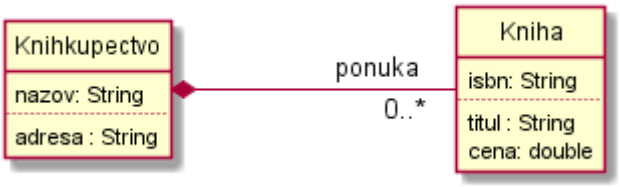
V prvej verzii budeme informácie uchovávať zatiaľ len v pamäti, v zozname obchodov, ktorý naplníme dátami pre testovanie:

```
        obchody = new ArrayList<>();
        Kniha k1 = new Kniha("111", "Hobit", 6.5);
        Kniha k2 = new Kniha("222", "Pipi dlha pancucha", 7.5);
        Kniha k3 = new Kniha("333", "Mach a Sebestova", 7.5);
        Knihkupectvo m = new Knihkupectvo("Martinus");
        Knihkupectvo p = new Knihkupectvo("PantaRhei");
        m.getPonuka().add(k1);
        m.getPonuka().add(k2);
        p.getPonuka().add(k1);
        p.getPonuka().add(k3);
        obchody.add(p);
        obchody.add(m);
```

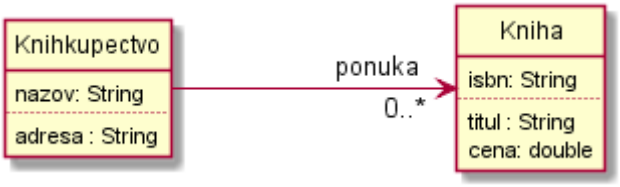
Implementácia metódy PUT pre modifikáciu údajov o knihách

Pri implementácii metódy PUT si však musíme položiť jednu dôležitú otázku:
Má zmena ceny knihy v jednom kníhkupectve spôsobiť zmenu aj vo všetkých ostatných?

Inými slovami: Je vzťah medzi kníhkupectvom a knihami v ponuke **kompozícia**



alebo **obyčajná asociácia**.

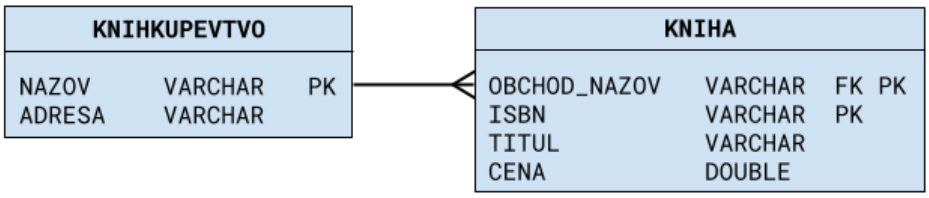


Poznámka: Od odpovede na túto otázku bude závisieť aj spôsob implementácie metód pre vkladanie a odstraňovanie kníh. (Pozri cvičenie 9.3 a 9.4)

Varianta 1 - Kompozícia: V tomto prípade má každé kníhkupectvo vo svojej ponuke vlastné kópie kníh.

Metódu PUT môžeme implementovať jednoducho tak, že pôvodnú knihu z ponuky kníhkupectva odstránime a novú inštanciu knihy vložíme do ponuky. Modifikácie údajov o knihe sa tak prejavia len v ponuke daného kníhkupectva.

Pre perzistentné uloženie informácií v databáze by mala mať kniha kľúč zložený z ISBN knihy a názvu kníhkupectva. Tabuľka `KNIHA` totiž v tomto prípade nepredstavuje samostatnú entitu.



Varianta 2 - Asociácia: V tomto prípade existuje pre každú knihu v pamäti len jedna inštancia. Na túto inštanciu odkazujú všetky kníhkupectvá, ktoré ju majú v ponuke.

Ak sú informácie udržiavané len v pamäti, je implementácia metódy PUT zložitejšia:
Pôvodnú referenciu na knihu neodstraňujeme, len v nej aktualizujeme zmenené dátové členy. Takto sa zmeny prejavia aj v ponuke všetkých kníhkupectiev, ktoré na ňu odkazujú.

Druhá možnosť je referenciu na knihu odstrániť a nahradiť odkazom na novú inštanciu. Treba to však urobiť pre všetky kníhkupectvá, ktoré ju ponúkali.

Pri perzistentnom uložení informácií v databáze sú `KNIHA` aj `KNIHKUPECTVO` dve plnohodnotné entity, medzi ktorými je vzťah N:N realizovaný prepojovacou tabuľkou:

Implementovať PUT môžeme teraz jednoducho pomocou metódy manažéra entít `merge`.

Implementácia varianty **'kompozícia'** s uložením údajov v pamäti je v projekte **SvetKnihV1pojo** na disku STU.

Implementácia varianty **'asociácia'** s uložením údajov v databáze je v projekte **SvetKnihV2db** na disku STU.

Varianty z pohľadu klienta

Keď klient zavolá metódu PUT pre URI `SvetKnih/{obchod}/{isbn}` zrejme očakáva, že zmení cenu len knihy len v danom obchode. V prípade implementácie ako asociácie, bude používateľ asi prekvapený, že sa pritom cena zmení vo všetkých obchodoch. O implementačných detailoch sa totiž z WADL súboru nič nedozvie.

Ak chceme, aby portál umožňoval meniť cenu knihy naraz vo všetkých obchodoch, nie je vhodné implementovať metódu PUT pre ponuku obchodu `SvetKnih/{obchod}/{isbn}`. Vhodnejší by bol priamy prístup ku knihe, napríklad prostredníctvom URI `SvetKnih/kniha/{isbn}`

Príklad 24: Portál Svet kníh - varianta s priamym prístupom ku knihám

Táto varianta služby realizuje vzťah medzi kníhkupectvom a knihami ako asociáciu medzi samostatnými zdrojmi, čo umožňuje pristupovať ku knihám nielen prostredníctvom kníhkupectva, ale aj priamo.

Implementuje nasledujúce metódy:

Pre `SvetKnih`

- GET vráti zoznam názvov obchodov vo formáte JSON.

Pre `SvetKnih/{obchod}`

- GET vráti všetky informácie o obchode a ponúkaných knihách vo formáte XML.

Pre `SvetKnih/{obchod}/{isbn}`

- GET vráti všetky informácie o knihe v ponuke obchodu vo formáte XML.

Pre `SvetKnih/kniha/{isbn}`

- PUT - umožní modifikovať informácie o knihe. Akceptuje informácie o knihe vo formáte XML.

Implementáciu služby by sme mohli doplniť tiež o GET metódy pre priame vyhľadávanie kníh:

Pre `SvetKnih/kniha`

- `GET` - dostane podreťazec ako parameter dotazu a vráti zoznam `isbn` všetkých kníh, ktoré majú podreťazec v názve. Formát JSON.

Pre `SvetKnih/kniha/{isbn}`

- `GET` - vráti všetky informácie o knihe vo formáte XML.
  Poznámka: Prístup ku knihám prostredníctvom URI obchodu `SvetKnih/{obchod}/{isbn}` teda nebude vôbec potrebný.

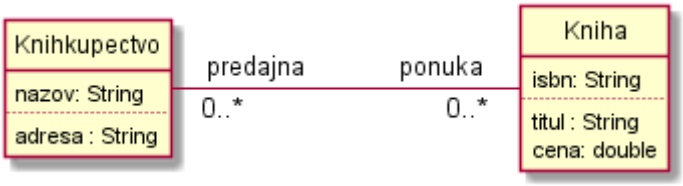
Implemenáciu príkladu 24 prenecháme na čitateľa, pozri cvičenie 9.2.

Príklad 25: Portál Svet kníh - varianta s obojstrannou asociáciou

Problém cyklických referencií

Pripomeňme, že asociácia medzi triedami `Knihkupectvo` a `Kniha` v príklade 24 je jednosmerná (pozri UML diagram špecifikácie služby v kapitole 9.2). Kníhkupectvo má zoznam ponúkaných kníh, ale kniha nemá žiadny dátový člen, ktorý by odkazoval na kníhkupectvá, kde je v ponuke. Klient, ktorý si informácie o knihe načíta priamo prostredníctvom URI `SvetKnih/kniha/{isbn}`, teda nevie priamo zistiť, v ktorých kníhkupectvách je v ponuke.

Ak by v objektovom modeli bola asociácia medzi kníhkupectvom a knihami obojsmerná, informáciu o predajniach, ktoré ju ponúkajú, by poskytovala priamo kniha. To môžeme dosiahnuť jednoducho doplnením dátovej triedy `Kniha` o zoznam kníhkupectiev.



Zmeny v implementácii prístupových metód, zdá sa, nie sú nutné.

Keď však takto upravenú službu spustíme a pomocou požiadavky GET pre `SvetKnih/{obchod}` sa pokúsime zistiť informácie o niektorom kníhkupectve, vráti služba kritickú výnimku:

```
HTTP Status 500 - Internal Server Error
```

Problém je v tom, že obojstranná asociácia vytvára cyklus referencií, ktorý by pri serializácii do XML alebo JSON vytvoril stromovú štruktúru nekonečnej hĺbky.

Poznámka: Bohužiaľ, chybová správa klienta integrovaného v prostredí NetBeans neuvádza, v čom je problém. Môžeme ho však reprodukovať jednoduchou aplikáciou, ktorá pomocou komponentu JAXB zapisuje dátové objekty do XML. Ak má knihkupectvo v ponuke nejakú knihu, jeho zápis do XML zlyhá s chybovou správou: `A cycle is detected in the object graph.`

Riešenie: Potrebné je zamedziť serializáciu referencovaných objektov z jednej strany asociácie pomocou anotácie `@XmlTransient`:

```
@XmlTransient
@ManyToMany
private List<Kniha> ponuka;
```

Teraz však reprezentácia kníhkupectva, ktorú vracia metóda GET pre URI `SvetKnih/{obchod}` neobsahuje ponuku kníh.

Poskytnúť zoznam kníh, ktoré ponúka dané kníhkupectvo, by mohla ďalšia GET metóda pre špeciálne URI, napr. `SvetKnih/{obchod}/ponuka`. Stačí, ak vráti zoznam ISBN kníh v ponuke. K detailným informáciám o konkrétnej knihe má klient prístup prostredníctvom URI `SvetKnih/kniha/{isbn}`.

Riešenie pomocou virtuálnej dátovej položky

Virtuálna dátová položka (*virtual property*) je dátová položka, ktorej stav je prístupný len prostredníctvom prístupových metód (*getter* a *setter*) rovnako ako štandardné dátové položky *java-bean* triedy. Na rozdiel od nich, jej stav však nie je uložený vo vlastnom dátovom člene, ale prístupová get-metóda si ho vypočíta pri každom zavolaní. Odvodzuje si ho zo stavu ostatných dátových členov.

V triede `Knihkupectvo` môžeme implementovať get-metódou pre virtuálnu položku poskytujúcu zoznam ISBN kníh v ponuke takto:

```
@XmlElementWrapper(name = "ponuka")
@XmlElement(name = "isbn")
public List<String> getIsbnList() {
   List<String> isbnList = new ArrayList<>();
   for(Kniha k: ponuka) {
      isbnList.add(k.getIsbn());
   }
   return isbnList;
}
```

Aby sa do XML serializovali aj virtuálne položky, je potrebné použiť anotáciu triedy. `@XmlAccessorType(XmlAccessType.PROPERTY)`:

```
@Entity
@XmlRootElement
@XmlAccessorType(XmlAccessType.PROPERTY)
public class Knihkupectvo implements Serializable {
```

Poznámka: Pri použití voľby `XmlAccessType.PROPERTY` treba presunúť všetky JAXB anotácie z dátových členov na getter metódy.

Implementáciu príkladu 25 s využitím virtuálnej dátovej položky nájdete v projekte **SvetKnihV3** na disku STU.

Cvičenia

Cvičenie 9.1

Reimplementujte projekt **SvetKnihV1pojo** (príklad 23, varianta 1 - kompozícia) pričom, informácie o knihách a kníhkupectvách budú uložené persistentne v relačnej databáze.

Cvičenie 9.2 Implementácia príkladu 24

Implementujte variantu portálu SvetKnih s priamym prístupom ku knihám tak, že informácie o knihách a kníhkupectvách budú

a) uchovávané len v pamäti servera,

b) uložené persistentne v relačnej databáze.

Cvičenie 9.3 Odstánenie knihy z ponuky kníhkupectva

Požiadavkou DELETE pre URI `SvetKnih/{obchod}/{isbn}` by klient mohol použiť na odstránenie knihy z ponuky kníhkupectva. Doplňte túto metódu do implementácií služby z príkladu 23 resp. cvičení 9.1 a 9.2.

Cvičenie 9.4 Pridanie existujúcej knihy ponuky kníhkupectva

Vo variante služby s priamym prístupom ku knihám (príklad 24) môže klient použiť metódu PUT aj na vytvorenie novej knihy v portáli kníh. Nová kniha však ešte nebude v ponuke žiadneho kníhkupectva.

Navrhnite a implementujte ďalšie rozšírenie služby, tak, aby poskytovala metódu aj na zaradenie existujúcej knihy do ponuky kníhkupectva. Implementujte tiež metódy pre odstránenie knihy z ponuky kníhkupectva, ako aj na úplné odstránenie knihy z portálu.

Navrhnite tiež spôsob, ktorý by aj vo variantách služby z príkladu 23 umožňoval klientom pridať do ponuky kníhkupectva knihu, ktorá existuje už v ponuke iného kníhkupectva.

10 HATEOAS

V predchádzajúcej časti sme si ilustrovali viacero problémov, s ktorými sa môžeme stretnúť pri návrhu, implementácii a používaní REST služby poskytujúcej prístup komplexnému systému navzájom asociovaných zdrojov. Aby sme sa problémom vyhli, implementovali sme GET metódy tak, že reprezentácia zdroja s odkazmi na ďalšie, asociované zdroje neobsahuje celú reprezentáciu asociovaných zdrojov, ale len ich identifikátory. Na základe týchto identifikátorov si môže klient vytvoriť URI asociovaných zdrojov a pristupovať k nim priamo.

Elegantnejšie riešenie by bolo poslať v odpovedi namiesto identifikátora asociovaného zdroja priamo jeho URI. URI asociovaných objektov môžeme vložiť do hlavičky http správy ako link. Asociované zdroje, ani ich identifikátory by sa do obsahu správy (vo formáte XML resp. JSON) vôbec neserializovali.

Príklad 26: Portál Svet kníh - varianta s použitím štýlu HATEOAS

V príklade 25 by sme mohli GET metódu pre URI `SvetKnih/{obchod}` implementovať nasledovne:

```
@GET
@Path("{obchod}")
@Produces(MediaType.APPLICATION_XML)
public Response getObchod(@Context UriInfo uriInfo,
        @PathParam("obchod") String obchod) {
    Knihkupectvo kk = em.find(Knihkupectvo.class, obchod);
    if (kk!=null && kk.getPonuka()!=null) {
        Response.ResponseBuilder rb = Response.ok(kk);
        for (Kniha k : kk.getPonuka()) {
            rb = rb.link(uriInfo.getBaseUriBuilder()
                    .path("SvetKnih")
                    .path("kniha")
                    .path(k.getIsbn()).build()
                    , "ponuka");
        }
        return rb.build();
    }
    return null;
}
```

Metóda vracia inštanciu `Response`, reprezentujúcu HTTP správu s odpoveďou. Dátový objekt kníhkupectva je serializovaný ako obsah správy.

Pre každú knihu v ponuke vloží do hlavičky správy link s URI pre priamy prístup ku knihe. Link okrem URI obsahuje aj ďalší identifikátor, ktorý vyjadruje vzťah k referencovanému zdroju. Tu sme použili `"ponuka"`, keďže ide o knihu v ponuke kníhkupectva.

Poznámka: Link je možné vložiť aj priamo do správy vo formáte XML. Pozri napríklad spring hateoas tutorial[19].

Rovnakým spôsobom môžeme upraviť aj ďalšie GET metódy:

GET metódu pre `SvetKnih/kniha/{isbn}`, ktorá vracia reprezentáciu knihy. Tu do hlavičky odpovede vložíme URI kníhkupectiev, v ktorých je kniha v ponuke.

GET metóda pre `SvetKnih` vráti v hlavičke URI všetkých obchodov, namiesto zoznamu názvov obchodov v obsahu správy.

GET metóda pre `SvetKnih/kniha` vráti v hlavičke URI hľadaných kníh, namiesto zoznamu ich ISBN v obsahu správy.

HATEOAS - Hypermedia as the Engine of Application State.

Štýl implementácie REST služby, ktorý sme tu použili, sa nazýva HATEOAS. Oddeluje klienta od serveru spôsobom, ktorý umožňuje serveru nezávislé rozširovanie funkcionality. Rieši zároveň problém s cyklickými referenciami. Klientovi umožňuje postupne pristupovať k asociovaným objektom ako samostatným zdrojom tak, že do obsahu správy vo formáte XML (resp. JSON) zapíše len dátové členy jednoduchých dátových typov. Asociované objekty predstavujú samostatné zdroje, ich URI sa posiela ako link. Reprezentáciu asociovaného zdroja si musí klient následne vyžiadať GET požiadavkou pre jeho URI.

Implementáciu služby portálu SvetKníh s použitím štýlu HATEOAS nájdete v projekte **SvetKnihV4** na disku STU.

[19] https://www.baeldung.com/spring-hateoas-tutorial

Implementácia klienta

Projekt **SvetKnihV4klient** obsahuje jednoduchú klientskú aplikáciu, ktorá nájde všetky kníhkupectvá, pre každé vypíše jeho názov a všetky informácie o knihách v ponuke.

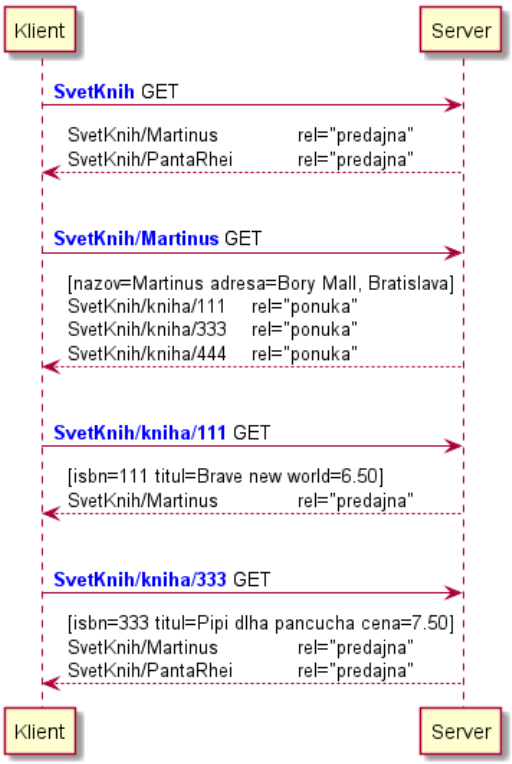
```
Client client = ClientBuilder.newClient();
Response resp = client
    .target("http://localhost:8080/SvetKnihV4/resources")
    .path("SvetKnih")
    .request(MediaType.TEXT_PLAIN)
    .get(Response.class);

for(Link l: resp.getLinks()) {
    Response resp2 = client
        .target(l)
        .request(MediaType.APPLICATION_XML)
        .get(Response.class);
    String obchod = resp2.readEntity(String.class);
    System.out.println(obchod);

    for(Link l2: resp2.getLinks()) {
        String kniha = client
            .target(l2)
            .request(MediaType.APPLICATION_XML)
            .get(String.class);
        System.out.println(kniha);
    }
}
```

Klient pošle najprv GET požiadavku pre koreňový zdroj `SvetKnih`. Návratovú správu načíta do inštancie `Response`, aby mal prístup k hlavičke, ktorá obsahuje URI asociovaných zdrojov t.j. knihkupectiev.

Následne pre každé kníhkupectvo vyšle GET požiadavku a odpoveď opäť načíta do inštancie `Response`. Odpoveď teraz obsahuje reprezentáciu kníhkupectva (názov, adresu...) a v hlavičke má URI asociovaných zdrojov, teda kníh v ponuke.

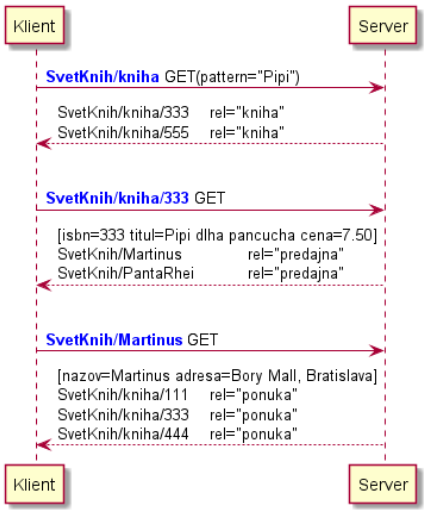
Ďalšou GET požiadavkou získa jej reprezentáciu knihy, ktorú načíta tentokrát ako už textový reťazec.

Celú komunikáciu medzi klientom a serverom ilustruje sekvenčný diagram na obr. 2



Obr. 2: Klient, ktorý číta informácie o kníhkupectvách a ponúkaných knihách

Portál SvetKnih však umožňuje medzi zdrojmi navigovať aj inak. Napríklad klient, ktorý si chce kúpiť konkrétnu knihu, si ju môže najprv vyhľadať podľa názvu, zistiť si jej cenu a následne aj adresu kníhkupectva, kde ju možno kúpiť. Celú komunikáciu zobrazuje sekvenčný diagram 3

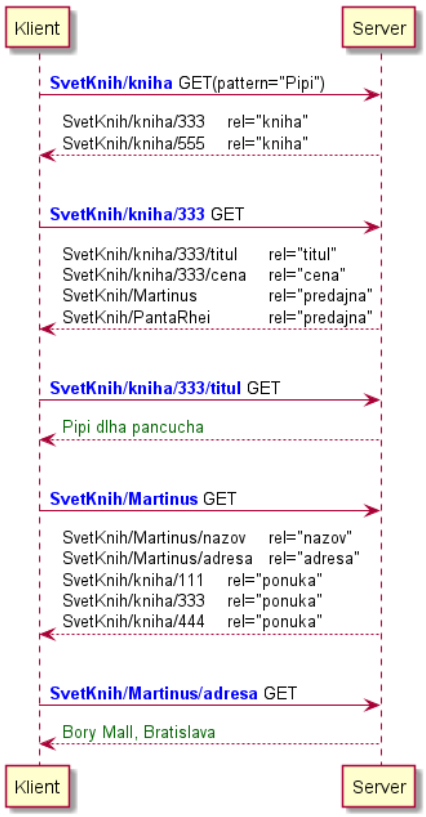


Obr. 3: Klient hľadajúci konkrétnu knihu

Pripomeňme, že WADL súbor neobsahuje popis vnútornej štruktúry dát posielaných vo formáte XML alebo JSON. Ak táto štruktúra nie je známa pri tvorbe klientskej aplikácie, nemôžeme pre ne vytvoriť vhodnú dátovú triedu, umožňujúcu pohodlný prístup k jednotlivým dátovým položkám (cene , adrese...).

Štýl HATEOAS môžeme však aplikovať aj na jednoduché dátové členy. Štrukturované dáta nie sú potom reprezentované vo formáte XML resp. JSON, ale každý dátový člen (nielen štrukturovaný) je prístupný ako samostatný zdroj. Jeho URI sa posiela ako link, pričom názov dátovej položky je identifikátor vzťahu. Posielajú sa teda len textové správy. A tak klient už nemusí mať vopred žiadne vedomosti o štruktúre dát spravovaných serverom. Odhaluje ju postupne na základe linkov v odpovediach od servera.

Po úprave služby by komunikácia s klientom, ktorý si chce kúpiť konkrétnu knihu, vyzerala takto:



Obr. 4

Cvičenia

Cvičenie 10.1 Implementácia klientov portálu Svet kníh

Implementujte klientskú aplikáciu služby z príkladu 26 (projekt **SvetKnihV4**), ktorá vyhľadá kníhkupectvo, v ktorom majú knihu o Pipi dlhej pančuche. Komunikácia by mala prebiehať tak, ako zobrazuje sekvenčný diagram na obrázku 3.

Cvičenie 10.2

Rovnako ako v príklade 25, umožňuje aj služba z príkladu 26 klientom pridať novú knihu do portálu. Nová kniha ale nebude v ponuke žiadneho kníhkupectva.

Navrhnite a implementujte aj pre túto variantu služby rozšírenie, ktoré by umožňovalo klientom pridať do ponuky kníhkupectva knihu, ktorá existuje už v portáli.

Navrhnite a implementujte pre túto variantu služby rozšírenie, ktoré by umožňovalo klientom aj odstrániť knihu z ponuky kníhkupectva.

Cvičenie 10.3 Implementácia portálu Svet kníh

Implementujte službu portálu SvetKnih, tak že štýl HATEOAS aplikujete na všetky dátové členy tried `Knihkupectvo` a `Kniha`.

Implementujte tiež klientskú aplikáciu, ktorá vyhľadá informácie o knihe `´Pipi dlhá pančucha´` a obchode, kde ju možno kúpiť. Komunikácia medzi klientom a službou by mala prebiehať tak, ako zobrazuje sekvenčný diagram na obrázku 4.

Cvičenie 10.4 Implementácia služby jedálne

Reimplementujte službu jedálne z cvičenia 8.10 s použitím štýlu HATEOAS.

Implementujte tiež klientskú aplikáciu služby jedálne, ktorá zistí a vypíše všetky informácie o jedlách v ponuke na pondelok.

Literatúra

[1] B. Burke. *RESTful Java with JAX-RS 2.0*. O'Reilly, 2013.

[2] G. Gregory and C. Bauer. *Java Persistence with Hibernate*. Manning, 2015.

[3] D.R. Heffelfinger. *Jakarta EE Application Development: Build enterprise applications with Jakarta CDI, RESTful web services, JSON Binding, persistence, and security*. Packt Publishing, 2024.

[4] Eric Jendrock, Ricardo Cervera-Navarro, Ian Evans, Devika Gollapudi, Kim Haase, William Markito, and Chinmayee Srivathsa. The Java EE 6 Tutorial. https://docs.oracle.com/javaee/6/tutorial/doc/docinfo.html, 2013. Accessed: 2023-12-08.

[5] M. Keith, M. Schincariol, and M. Nardone. *Pro JPA 2 in Java EE 8: An In-Depth Guide to Java Persistence APIs*. Apress, 2018.

[6] D. Pilone and N. Pitman. *UML 2.0 in a Nutshell: A Desktop Quick Reference*. In a Nutshell (O'Reilly). O'Reilly Media, 2005.

[7] H. Subramanian and P. Raj. *Hands-On RESTful API Design Patterns and Best Practices: Design, develop, and deploy highly adaptable, scalable, and secure RESTful web APIs*. Packt Publishing, 2019.